TRỰC QUAN HÓA DỮ LIỆU

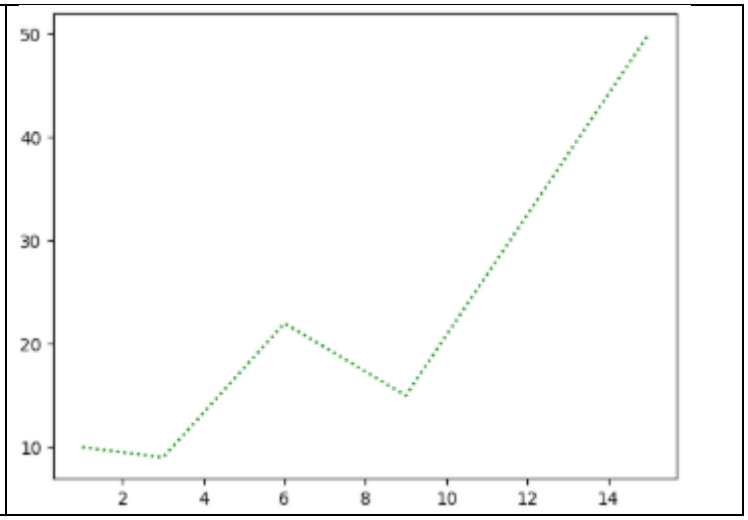
LAB1: Vẽ Biểu đồ Line

Bài 1. Vẽ biểu đồ Line nét chấm dotted và có màu xanh:

```
import matplotlib.pyplot as plt
import numpy as np

# Đỉnh trục  x
x_points = np.array([1,3,6,9,15])

# Đỉnh trục  y
y_points = np.array([10,9,22,15,50])
plt.plot(x_points, y_points, linestyle=':',
color='g')
plt.show()
```



Bài 2. Kết hợp vẽ 2 Lines để hiển thị kết quả sau:

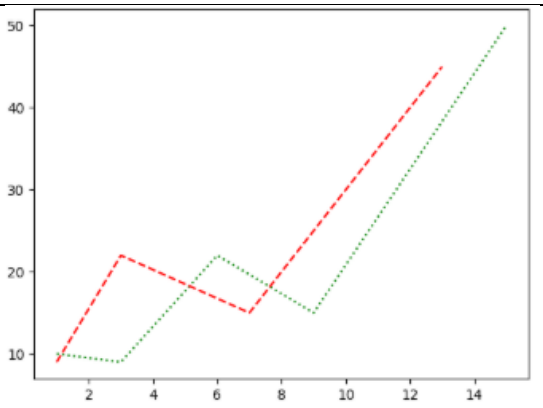
```
import matplotlib.pyplot as plt
import numpy as np

# Đỉnh trục  x
x_points = np.array([1,3,6,9,15])
x1_points = np.array([1,3,7,13])

# Đỉnh trục  y
y_points = np.array([10,9,22,15,50])
y1_points = np.array([9,22,15,45])

plt.plot(x_points, y_points, linestyle=':', color='g')
plt.plot(x1_points, y1_points, linestyle='--', color='r')

plt.show()
plt.show()
```

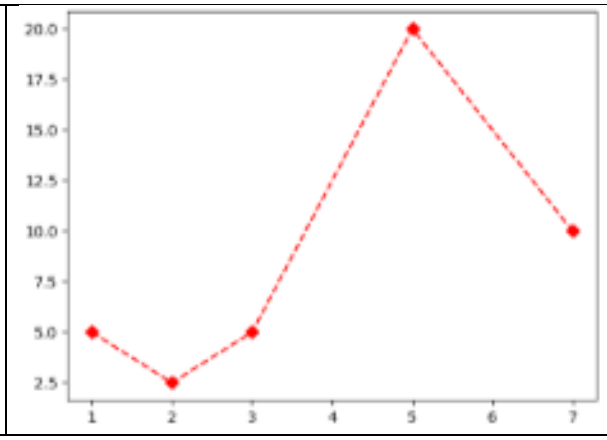


Bài 3. Vẽ Line có đỉnh hình vuông và nét dứt màu đỏ hiển thị kết quả sau:

```
import matplotlib.pyplot as plt
import numpy as np

# Đỉnh trục  x
x_points = np.array([1,2,3,5,7])

# Đỉnh trục  y
y_points = np.array([5,2.5,5,20,10])
plt.plot(x_points, y_points,'rD--')
plt.show()
```



BÀi 4.Vẽ biểu đồ sau để hiển thị sự tăng giảm của Giá nhiên liệu trong năm 2020:

- **[20,22,24,26,28,30] là các mảng tọa độ x**
- **[5,3,5,4,5,5], [4,4.5,5,3,4,7], [5,4,6,4,6,8] là các mảng tọa độ x**
- **color hiển thị màu đường đồ thị: blue, green, red**
- **label Chú thích đồ thi**

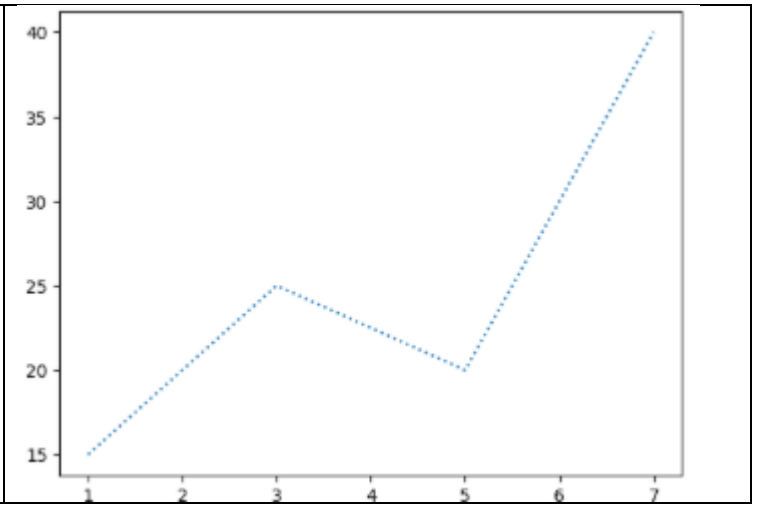
```
import numpy as np
import matplotlib.pyplot as plt
# Đỉnh trục  x
x_points = np.array([20, 22, 24, 26, 28, 30])
# Đỉnh trục  y
y_points = np.array([5, 3, 5, 4, 5, 5])
y1_points = np.array([4, 4.5, 5, 3, 4, 7])
y2_points = np.array([5, 4, 6, 4, 6, 8])

plt.plot(x_points, y_points, linestyle='-',
color='b', label='Xăng')
plt.plot(x_points, y1_points, linestyle='-',
color='g',  label='Dầu')
plt.plot(x_points, y2_points, linestyle='-',
color='r', label='Nhớt')
plt.xlabel('Giá nhiên liệu (VNĐ)')
plt.ylabel('Thời Gian (Tháng)')
plt.title('Biểu đồ nhiên liệu trong nước (năm
2020)')
plt.legend()
plt.show()
```



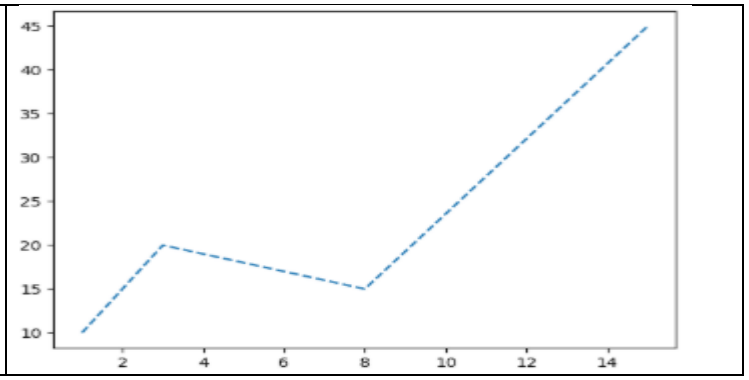
VD1: Vẽ biểu đồ Line trong python có dạng nét đứt bởi các dấu chấm (dotted):

```
import matplotlib.pyplot as plt
import numpy as np
# Đỉnh trục x
x_points = np.array([1, 3, 5, 7])
# Đỉnh trục y
y_points = np.array([15, 25, 20, 40])
plt.plot(x_points, y_points, linestyle = 'dotted')
plt.show()
```



VD2: Vẽ biểu đồ đường Line trong python có dạng nét đứt bởi các dấu gạch (dashed):

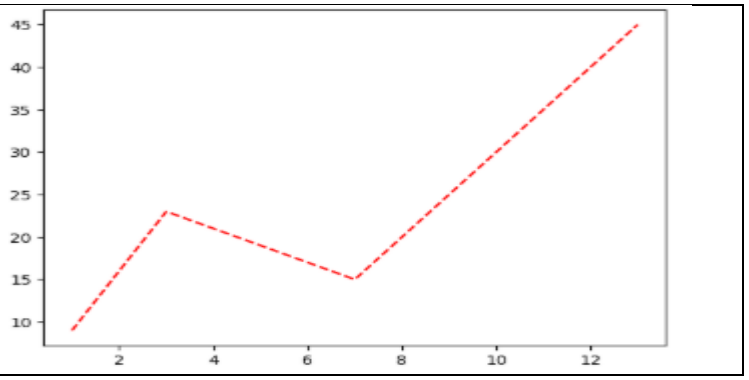
```
import matplotlib.pyplot as plt
import numpy as np
x_points = np.array([1, 3, 8, 15])
y_points = np.array([10, 20, 15, 45])
plt.plot(x_points, y_points, linestyle = 'dashed')
plt.show()
```



VD3: Cách thay đổi màu sắc của đường (Line) trong biểu đồ (Line Color)

- **Sử dụng từ khóa color hoặc c để thiết lập biểu thị màu sắc của đường.**
- **Tham số màu : r - red, g -green, b - blue, y - yellow, … Vẽ biểu đồ đường nét đứt dashed và có màu đỏ.**

```
import matplotlib.pyplot as plt
import numpy as np
x_points = np.array([1, 3, 7, 13])
y_points = np.array([9, 23, 15, 45])
plt.plot(x_points, y_points, linestyle='dashed',
color = 'r')
plt.show()
```

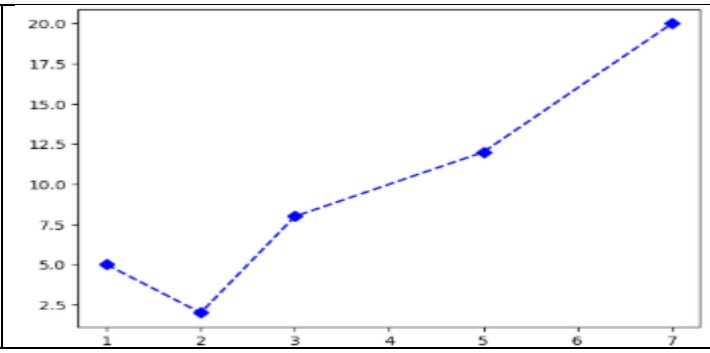


VD4: Cách vẽ có định dạng Đỉnh

- **Định dạng đường: - (đường thẳng), -- (đường nét đứt), nếu không truyền tham số là không vẽ đường.**
- **Định dạng đỉnh: * (đỉnh hình bông tuyết), D (đỉnh hình thoi), s (đỉnh hình vuông), go (chấm tròn), ...**

Ví dụ: Thêm tham số b tức là màu blue, D tức là đỉnh hình thoi và - tức là cạnh nét đứt.

```
import matplotlib.pyplot as plt
import numpy as np
x_points = np.array([1, 2, 3, 5, 7])
y_points = np.array([5, 2, 8, 12, 20])
plt.plot(x_points, y_points,'bD--')
plt.show()
```



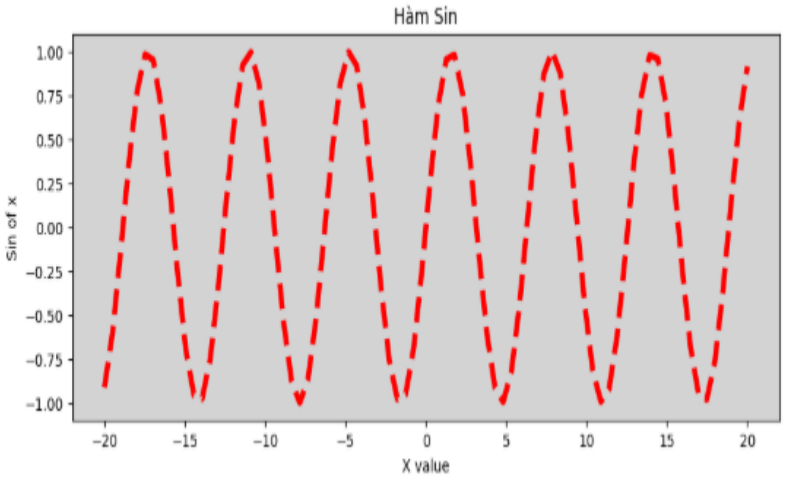
LAB2: Matplotlib Plot.

Bài 1. Vẽ biểu đồ hình sin cho thư viện numpy (np.linspace((-20,20,80), độ dày =4:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(-20, 20, 80)
y = np.sin(x)

# Vẽ biểu đồ
plt.figure(figsize=(10, 4))
plt.plot(x, y, 'r--',linewidth=4)

# Tùy chỉnh nền
plt.gca().set_facecolor('lightgray')
plt.title("Hàm Sin")
plt.xlabel("X value")
plt.ylabel("Sin of x")
plt.show()
```



Bài 2 .Vẽ biểu đồ line cho list các điểm:

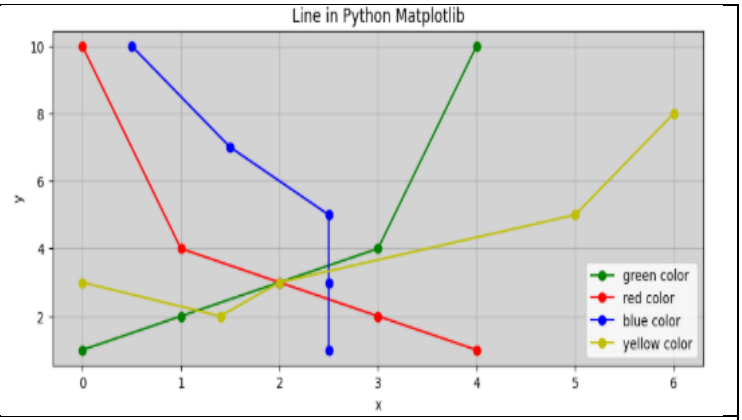
[0,1,2,3,4], [1,2,3,4,10]

[0,1,2,3,4], [10,4,3,2,1]

[2.5,2.5,2.5,1.5,0.5], [1,3,5,7,10]

[0,1.4,2,5,6], [3,2,3,5,8]

```
import matplotlib.pyplot as plt
import numpy as np
# Data
x1, y1 = [0, 1, 2, 3, 4], [1, 2, 3, 4, 10]
x2, y2 = [0, 1, 2, 3, 4], [10, 4, 3, 2, 1]
x3, y3 = [2.5, 2.5, 2.5, 1.5, 0.5], [1, 3, 5, 7, 10]
x4, y4 = [0, 1.4, 2, 5, 6], [3, 2, 3, 5, 8]
```




```
# Vẽ các đường với màu sắc khác nhau
plt.figure(figsize=(10, 4))
plt.plot(x1, y1, 'g-', marker='o', label='green color')
plt.plot(x2, y2, 'r-', marker='o', label='red color')
plt.plot(x3, y3, 'b-', marker='o', label='blue color')
plt.plot(x4, y4, 'y-', marker='o', label='yellow color')

plt.gca().set_facecolor('lightgray')
plt.title("Line in Python Matplotlib")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid()
plt.show()
```

Bài 3. Vẽ biểu đồ line cho list các điểm:

[0,1,4,6,5], [1,2,3,5,6]

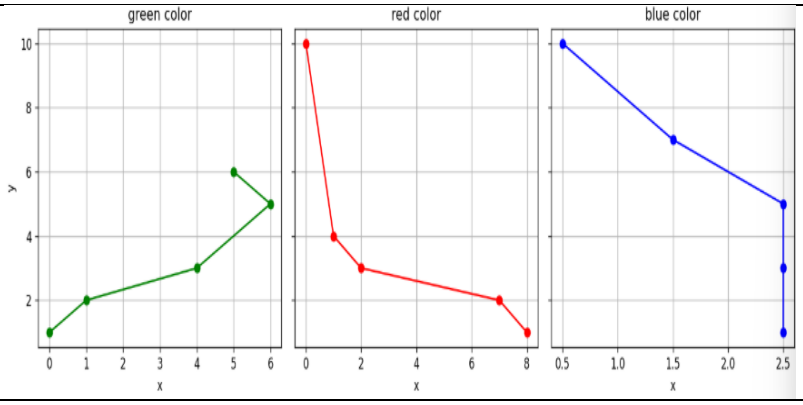
[0,1,2,7,8], [10,4,3,2,1]

[2.5,2.5,2.5,1.5,0.5], [1,3,5,7,10]

Tạo 3 ax là 3 biểu đồ:

```
import matplotlib.pyplot as plt

# Dữ liệu các đường
x1, y1 = [0,1,4,6,5], [1, 2, 3, 5, 6]
x2, y2 = [0,1,2,7,8], [10, 4, 3, 2, 1]
x3, y3 = [2.5, 2.5, 2.5, 1.5, 0.5],
[1, 3, 5, 7, 10]
```




```
# Tạo figure với 3 subplot, cùng chia sẻ trục Y
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12, 4), sharey=True, dpi=120)
# Vẽ từng đường trên từng subplot với màu sắc và marker phù hợp
ax1.plot(x1, y1, 'g-', marker='o')
ax1.set_title("green color")
ax1.set_xlabel("x")
ax1.set_ylabel("y")

ax2.plot(x2, y2, 'r-', marker='o')
ax2.set_title("red color")
ax2.set_xlabel("x")

ax3.plot(x3, y3, 'b-', marker='o')
ax3.set_title("blue color")
ax3.set_xlabel("x")

# Hiển thị lưới trên tất cả subplot
for ax in [ax1, ax2, ax3]:
    ax.grid(True)

# Sắp xếp layout hợp lý
plt.tight_layout()
plt.show()
```

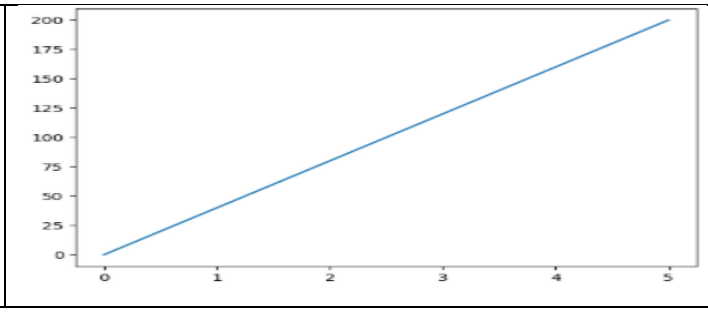
VD1: Cách sử dụng hàm plot để vẽ biểu đồ.

```
# Vẽ x và y sử dụng kiểu đường thẳng và màu sắc mặc định.
# Vẽ x và y sử dụng kiểu loại của điểm Markers là hình tròn.
# Vẽ y sử dụng x như một mảng index mặc định : 0,1,2...N-1.
```

Nhưng kiểu loại là dấu '+' Cho y = f(x) trong biểu đồ bởi hàm plot(x, y).

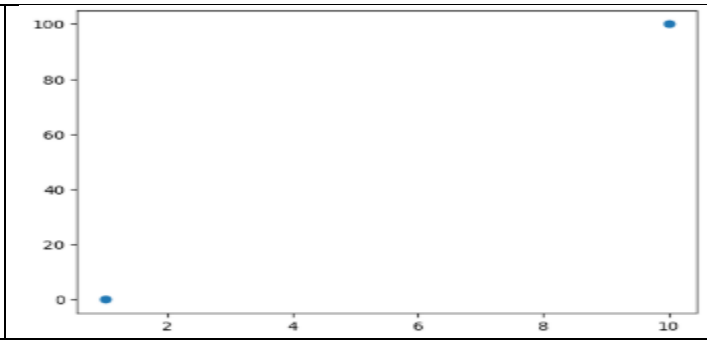
Ví dụ vẽ biểu đồ từ điểm có tọa độ (0,0) tới điểm có tọa độ (5,200) như sau:

```
import matplotlib.pyplot as plt
import numpy as np
x_points = np.array([0,5])
y_points = np.array([0,200])
plt.plot(x_points, y_points)
plt.show()
```



VD2: Vẽ các điểm và không vẽ đường thẳng trong biểu đồ trong python: Hàm này được dùng cho trường hợp chỉ vẽ các điểm, không vẽ đường thẳng trong biểu đồ. Tham số 'o' là các điểm được biểu diễn bởi các hình tròn.

```
import matplotlib.pyplot as plt
import numpy as np
x_points = np.array([1,10])
y_points = np.array([0,100])
plt.plot(x_points, y_points, 'o')
plt.show()
```

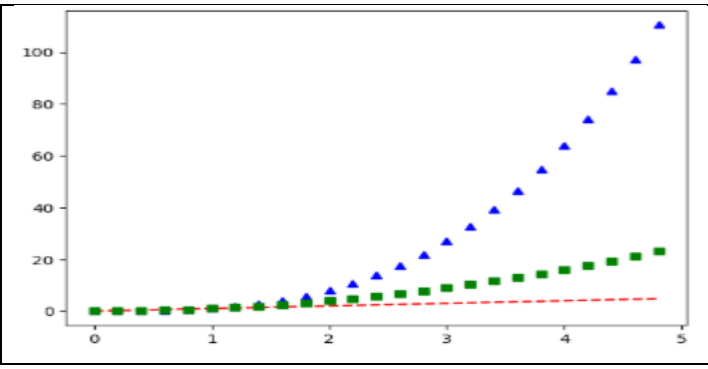


VD3: Vẽ đồ thị định dạng Plot:

Phương thức axis () cho phép đặt chế độ xem cho các trục theo xmin, xmax, ymin và ymax.
Chuỗi định dạng 'gs' ở đây cho chúng ta các ô vuông màu xanh lá cây. Tương tự, ta cũng có thể vẽ các quả cầu màu xanh lá cây, đỏ và xanh lam, hình vuông và hình tam giác.

```
import matplotlib.pyplot as plt
import numpy as np

t=np.arange(0,5,0.2)
# vẽ đồ thị
plt.plot(t,t, 'r--',t,t**3, 'b^',t,t**2, 'gs')
plt.show()
```



LAB3: BAR CHART

BÀI TẬP 1:

Thông thường, một biểu đồ phải có chú thích của trục x và trục y theo đối tượng mình cần trực quan hóa

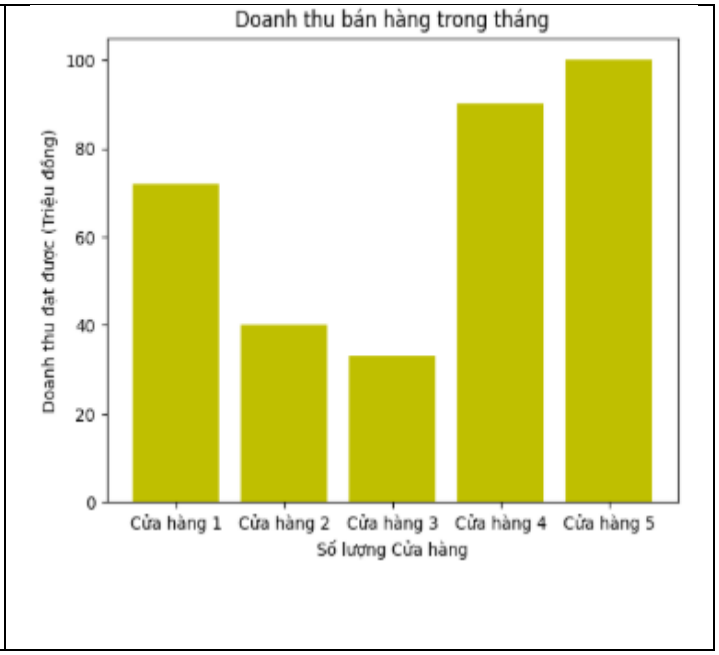
Ta có thể thêm Title và Axis Labels với các số cuộn trên trục x và các dấu trên trục y.

Trực quan hóa doanh thu (Triệu đồng) của các Cửa hàng trong tháng

```
import matplotlib.pyplot as plt
import numpy as np

marks=[72,40,33,90,100]
bars=('Cửa hàng 1', 'Cửa hàng 2', 'Cửa hàng 3', 'Cửa hàng 4', 'Cửa hàng 5')
y=np.arange(len(bars))

plt.bar(bars,marks,color='y')
plt.xticks(y,bars)
plt.title('Doanh thu bán hàng trong tháng')
plt.xlabel('Số lượng Cửa hàng')
plt.ylabel('Doanh thu đạt được (Triệu đồng)')
plt.show()
```



BÀI TẬP 2:Xem dữ liệu file tips.csv trong Data:

Viết chương trình python:

- **Show để xem các cột trong dữ liệu**

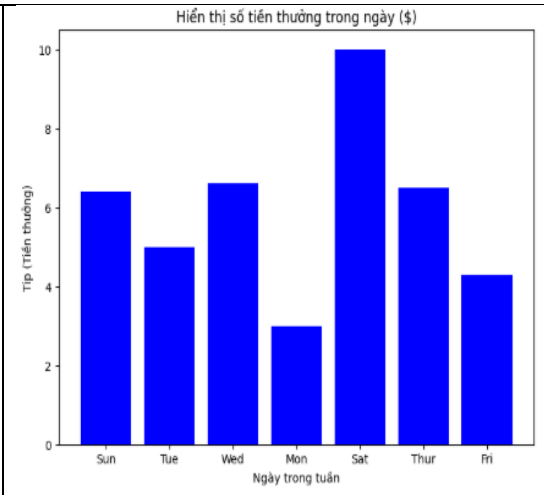
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	1
1	10.34	1.66	Male	No	Sun	Dinner	1
2	21.01	3.50	Male	No	Tue	Dinner	3
3	23.68	3.31	Male	No	Tue	Dinner	3
4	24.59	3.61	Female	No	Sun	Dinner	1
..	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	7
240	27.18	2.00	Female	Yes	Sat	Dinner	7
241	22.67	2.00	Male	Yes	Sat	Dinner	7
242	17.82	1.75	Male	No	Sat	Dinner	7
243	18.78	3.00	Female	No	Thur	Dinner	5

- **Hãy trực quan hóa dữ liệu Bar chart hiển thị số ngày trong tuần có được tiền thưởng (tip)**

```
import matplotlib.pyplot as plt
import numpy as np

marks = [6.4, 5, 6.6, 3, 10, 6.5, 4.3]
bars = ['Sun', 'Tue', 'Wed', 'Mon', 'Sat', 'Thur', 'Fri']
y = np.arange(len(bars))

plt.figure(figsize=(8, 6))
plt.bar(bars, marks, color='blue')
plt.xticks(rotation=0)
plt.title('Hiển thị số tiền thưởng trong ngày ($)')
plt.xlabel('Ngày trong tuần')
plt.ylabel('Tip (Tiền thưởng)')
plt.show()
```



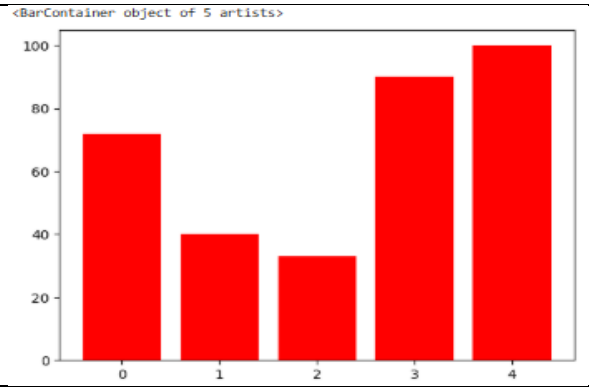
A. Vẽ biểu đồ cột (Matplotlib Bar).

Biểu đồ thanh trong Python, còn được gọi là biểu đồ thanh, biểu thị cách một biến số liên quan đến một biến phân loại.

Ví dụ 1: Vertical Bar Plot (Biểu đồ thanh dọc)

```
import matplotlib.pyplot as plt
import numpy as np

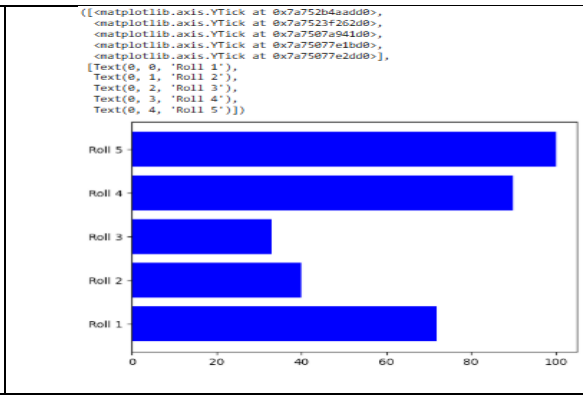
marks=[72,40,33,90,100]
bars=('Roll 1', 'Roll 2', 'Roll 3', 'Roll 4', 'Roll 5')
y=np.arange(len(bars))
plt.bar(y,marks,color='r')
```



Ví dụ 2: Horizontal Bar Plot (Biểu đồ thanh ngang)

```
import matplotlib.pyplot as plt
import numpy as np

marks=[72,40,33,90,100]
bars=('Roll 1', 'Roll 2', 'Roll 3', 'Roll 4', 'Roll 5')
y=np.arange(len(bars))
plt.barh(y,marks,color='b')
plt.yticks(y,bars)
```



Ví dụ 3: Bar thể hiện Nhóm tuổi của Male và Female

```
import matplotlib.pyplot as plt
import numpy as np

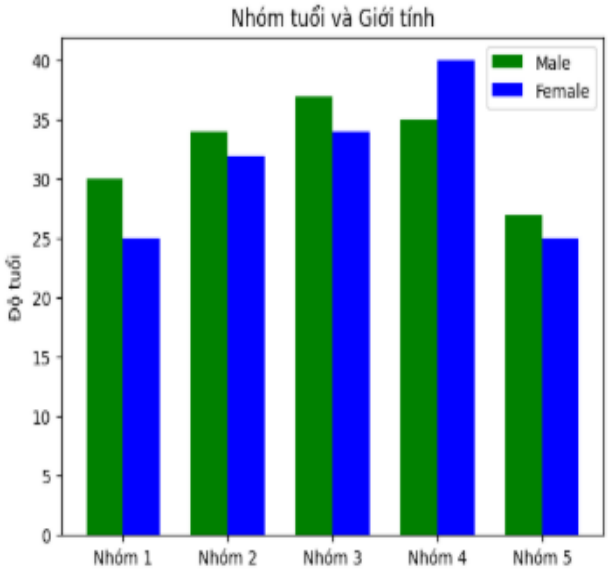
Male = [30, 34, 37, 35, 27]
Female = [25, 32, 34, 40, 25]
group = ['Nhóm 1', 'Nhóm 2', 'Nhóm 3', 'Nhóm 4',
'Nhóm 5']

# Lấy cột từ group
x = np.arange(len(group))
# độ rộng của bars
width = 0.35

# Vẽ biểu đồ cột
fig, ax = plt.subplots()
rects1 = ax.bar(x - width/2, Male, width, color='g',
label='Male')
rects2 = ax.bar(x + width/2, Female, width, color='b',
label='Female')

# Gán Label và Title
ax.legend()
ax.set_ylabel('Độ tuổi')
ax.set_title('Nhóm tuổi và Giới tính')
ax.set_xticks(x)
ax.set_xticklabels(group)
```

```
[Text(0, 0, 'Nhóm 1'),
 Text(1, 0, 'Nhóm 2'),
 Text(2, 0, 'Nhóm 3'),
 Text(3, 0, 'Nhóm 4'),
 Text(4, 0, 'Nhóm 5')]
```



LAB4: SCATTER PLOT MATPLOTLIB

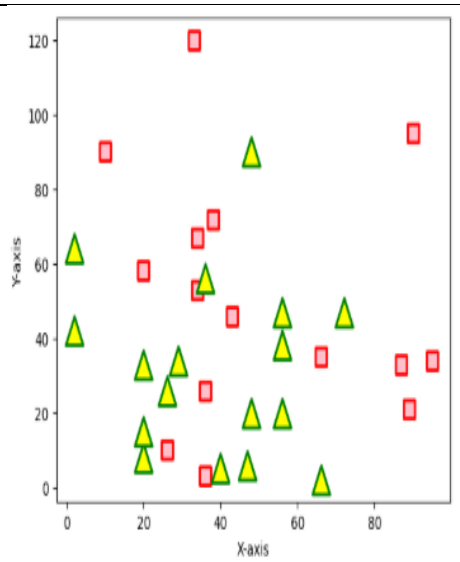
BÀI TẬP 1: Cho 2 dataset:

```
# dataset-1
x1 = [89, 43, 36, 36, 95, 10, 87,
      66, 34, 38, 20, 26, 34, 90, 33]
y1 = [21, 46, 3, 26, 34, 90, 33, 35,
      53, 72, 58, 10, 67, 95,120]
# dataset2
x2 = [26, 29, 48, 20, 56, 2, 48, 20, 56,
      36, 66, 72, 20, 56, 2, 47, 40]
y2 = [26, 34, 90, 33, 38, 42, 20, 8,
      20, 56, 2, 47, 15, 47, 64, 6, 5]
```

Hãy vẽ biểu đồ scatter các điểm như biểu đồ sau:

```
import matplotlib.pyplot as plt
# dataset của x1,y1 và x2,y2
x1 = [89, 43, 36, 36, 95, 10, 87, 66, 34, 38, 20, 26, 34, 90, 33]
y1 = [21, 46, 3, 26, 34, 90, 33, 35, 53, 72, 58, 10, 67, 95, 120]

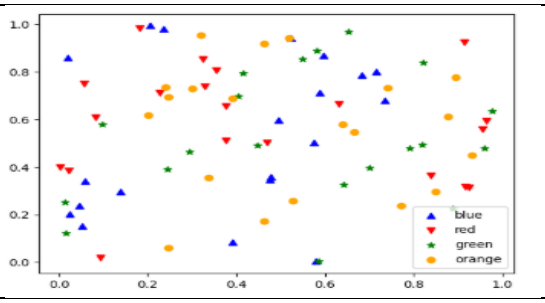
x2 = [26, 29, 48, 20, 56, 2, 48, 20, 56, 36, 66, 72, 20, 56, 2, 47, 40]
y2 = [26, 34, 90, 33, 38, 42, 20, 8, 20, 56, 2, 47, 15, 47, 64, 6, 5]
# Tạo các nốt với các thuộc tính sau:
plt.scatter(x1, y1, c ="pink",
            linewidths = 2,
            marker ="s",
            edgecolor ="red",
            s = 100)
plt.scatter(x2, y2, c ="yellow",
            linewidths = 2,
            marker ="^",
            edgecolor ="green",
            s = 250)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```



BÀI TẬP 2: Vẽ biểu đồ scatter như biểu đồ sau, có 4 màu với các kiểu khác nhau:

Cho n1, n2, n3, n4: với random(20)

```
import matplotlib.pyplot as plt
import numpy as np
# Số lượng điểm cho mỗi nhóm
n1, n2, n3, n4 = 20, 20, 20, 20
```




```
# Tạo dữ liệu ngẫu nhiên
x1, y1 = np.random.rand(n1), np.random.rand(n1)
x2, y2 = np.random.rand(n2), np.random.rand(n2)
x3, y3 = np.random.rand(n3), np.random.rand(n3)
x4, y4 = np.random.rand(n4), np.random.rand(n4)
# Vẽ scatter plot
plt.scatter(x1, y1, color='blue', marker='^', label='blue')
plt.scatter(x2, y2, color='red', marker='v', label='red')
```

```
plt.scatter(x3, y3, color='green', marker='*', label='green')
plt.scatter(x4, y4, color='orange', marker='o', label='orange')

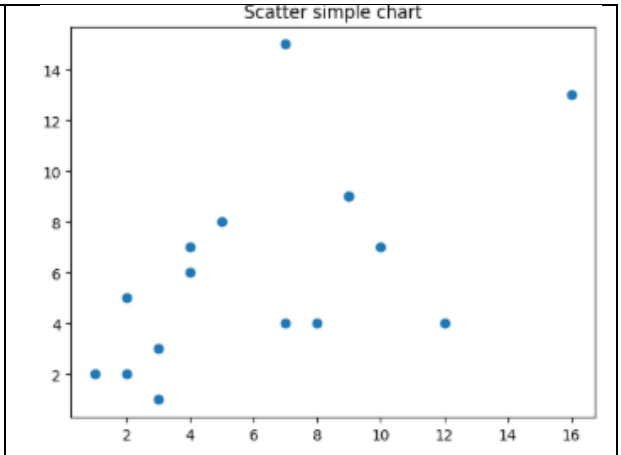
plt.legend()
plt.show()
```

Ví dụ 1: Trực quan hóa các nốt theo x và y

```
# Ví dụ 1
import matplotlib.pyplot as plt
import numpy as np
# data hiển thị trên plots
x = np.array ([3, 1, 3, 12, 2, 4, 4, 2, 8, 7, 9, 16, 7, 5, 9, 10])
y = np.array ([3, 2, 1, 4, 5, 6, 7, 2, 4, 4, 9, 13, 15, 8, 9, 7 ])

# Đồ thị đơn giản scatter chart
plt.scatter(x, y)

# Title
plt.title("Scatter simple chart")
plt.show()
```

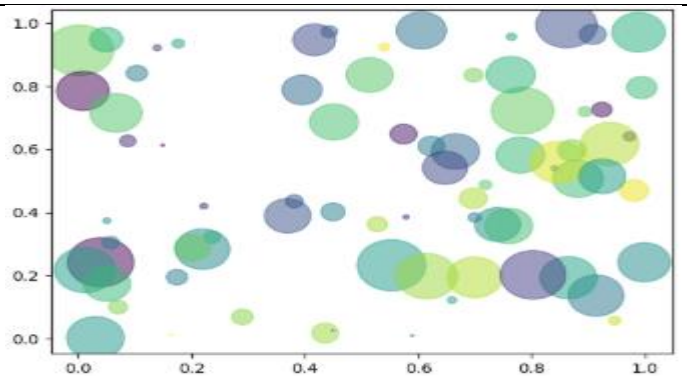


Ví dụ 2: Tạo hàm ngẫu nhiên với 75 nốt với màu sắc ngẫu nhiên hiển thị alpha=0.5. Vẽ biểu đồ thể hiện scatter cho các nốt

```
import numpy as np
import matplotlib.pyplot as plt

# hàm khởi tạo các bộ sinh số ngẫu nhiên
(random generator).

# Biến số trong seed thường là một số nguyên
không âm 32 bit.
np.random.seed(13499202)
```




```
# Gán số lượng N (Bong bóng)
N = 75
x = np.random.rand(N)
y = np.random.rand(N)

# Gán màu sắc random
```

```
colors = np.random.rand(N)

# Gán bán kính random cho các điểm từ 0 đến 40 (độ lớn Bình Phương)
area = (40 * np.random.rand(N))**2

# in theo x và y (diện tích và màu sắc)
plt.scatter(x, y, s=area, c=colors, alpha=0.5)
plt.show()
```

Ví dụ 3: Cho các thuộc tính:

Màu c ="yellow",

Đọ dày của viền linewidths = 2,

Điểm kiểu vuông marker ="s",

Biên của điểm edgecolor ="green",

Các điểm có kích thước 100

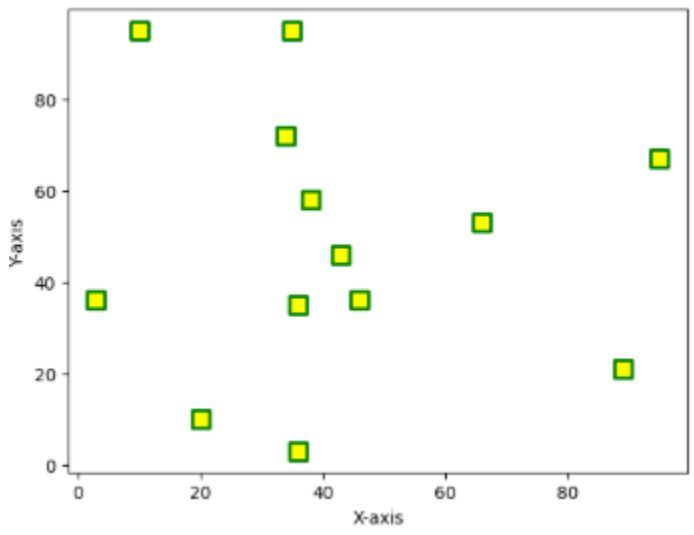
Hãy vẽ biểu đồ scatter các điểm?

```
import matplotlib.pyplot as plt

# dataset của x và y
x = [89, 43, 36, 36, 95, 10, 66, 34, 38, 20, 46, 3,
35]
y = [21, 46, 3, 35, 67, 95, 53, 72, 58, 10, 36, 36,
95]

# Tạo các nốt với các thuộc tính sau:
plt.scatter(x, y, c ="yellow",
        linewidths = 2,
        marker ="s",
        edgecolor ="green",
        s = 100)

plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```



LAB 5: PIE CHART

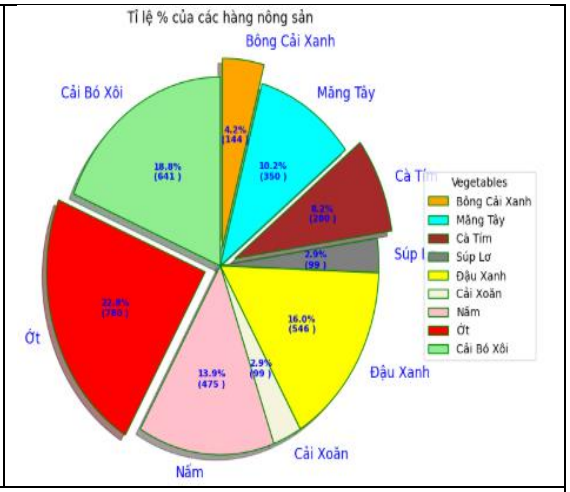
BÀI TẬP: Vẽ Biểu đồ Pie Chart thể hiện tỉ lệ % của các hàng nông sản đã bán theo bảng dữ liệu sau.

ID	Nông sản	Đơn vị đã Bán
1009	Bông cải xanh	145
1007	Măng tây	350
1003	Cà tím	280
1005	Súp lơ	100
1006	Đậu xanh	547
1001	Cải xoăn	100
1004	Nấm	475
1008	Ớt	780
1002	Cải bó xôi	642

```
import numpy as np
import matplotlib.pyplot as plt

# Dữ liệu từ bảng
vegetables = ['Bông Cải Xanh', 'Măng Tây', 'Cà Tím', 'Súp Lơ',
'Đậu Xanh', 'Cải Xoăn', 'Nấm', 'Ớt', 'Cải Bó Xôi']
data = [145, 350, 280, 100, 547, 100, 475, 780, 642]

# Tạo explode để làm nổi bật một số phần
explode = (0.1, 0.0, 0.1, 0.0, 0.0, 0.0, 0.0, 0.1, 0.0)
```




```
# Tạo màu sắc
colors = ("orange", "cyan", "brown", "grey", "yellow", "beige", "pink", "red", "lightgreen")

# Thuộc tính Wedge (vùng)
wp = {'linewidth': 1, 'edgecolor': "green"}

# Tạo tỉ lệ
def func(pct, allvalues):
    absolute = int(pct / 100. * np.sum(allvalues))
    return "{:.1f}%\n({:d} )".format(pct, absolute)

# Tạo plot
fig, ax = plt.subplots(figsize=(10, 7))
```

```
wedges, texts, autotexts = ax.pie(
    data,
    autopct=lambda pct: func(pct, data),
    explode = explode,
    labels = vegetables,
    shadow = True,
    colors = colors,
    startangle = 90,
    counterclock = False,
    wedgeprops = wp,
    textprops = dict (size=12, color="blue"))

# Thêm legend
ax.legend(
    wedges, vegetables,
    title = "Vegetables",
    loc = "center left",
    bbox_to_anchor = (1, 0, 0.5, 1))
plt.setp(autotexts, size=8, weight="bold")
ax.set_title("Tỉ lệ % của các hàng nông sản")
plt.show()
```

Ví dụ 1:

```
import matplotlib.pyplot as plt
import numpy as np

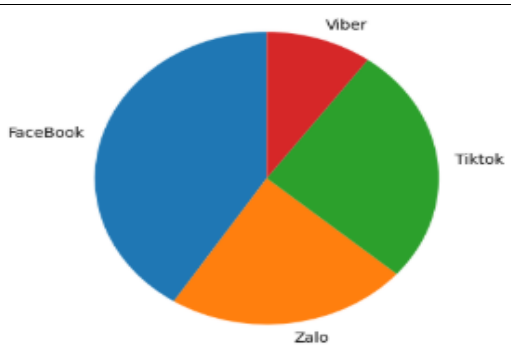
y = np.array([35, 25, 25, 15])
plt.pie(y)
plt.show()
```



Ví dụ 2: Gán nhãn theo từng vùng:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([45, 25, 29, 11])

# Gán nhãn
mylabels = ["FaceBook", "Zalo", "Tiktok", "Viber"]
plt.pie(y, labels = mylabels, startangle = 90)
plt.show()
```

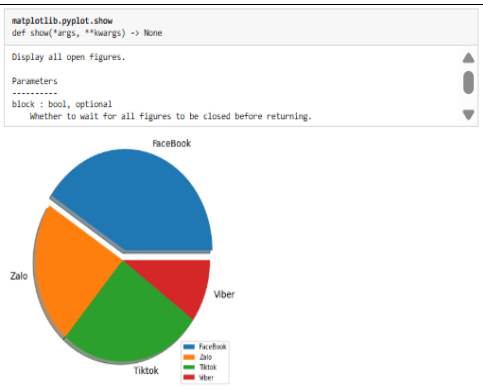


Ví dụ 3: Cho 1 vùng có độ hở theo vùng, gán shadow và gán nhãn:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([45, 25, 29, 11])
mylabels = ["FaceBook", "Zalo", "Tiktok", "Viber"]
# Gán khoảng trống
myexplode = [0.1, 0, 0, 0]

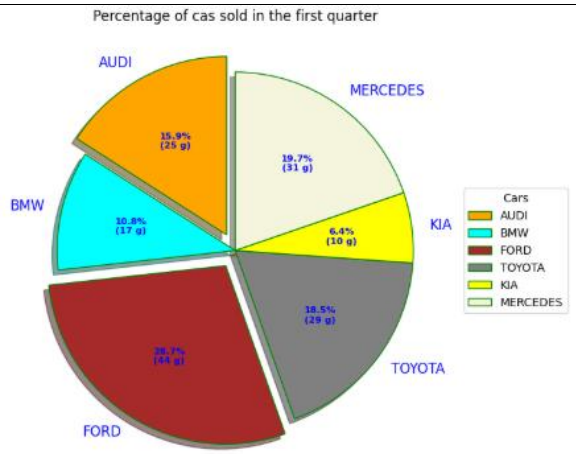
plt.pie(y, labels = mylabels, explode = myexplode, shadow = True)
plt.legend(loc='best', fontsize=7)
plt.show
```



Ví dụ 4: Tạo Biểu đồ Pie có chú thích tỉ lệ % của các loại xe bán được của một của hàng:

```
import numpy as np
import matplotlib.pyplot as plt

# Tạo dữ liệu
cars = ['AUDI', 'BMW', 'FORD', 'TOYOTA', 'KIA',
'MERCEDES']
data = [25, 17, 45, 29, 10, 31]
# Tạo explode data
explode = (0.1, 0.0, 0.1, 0.0, 0.0, 0.0)
```




```
# Tạo màu sắc
colors = ("orange", "cyan", "brown", "grey", "yellow", "beige")
# Thuộc tính Wedge (vùng)
wp = {'linewidth': 1, 'edgecolor': "green" }

# Tạo tỉ lệ
def func(pct, allvalues):
    absolute = int(pct / 100.*np.sum(allvalues))
    return "{:.1f}%\n({:d} g)".format(pct, absolute)

# Tạo plot
fig, ax = plt.subplots(figsize =(10, 7))
```

```
wedges, texts, autotexts = ax.pie(data,
                  autopct = lambda pct: func(pct, data),
                   explode = explode,
                      labels = cars,
                   shadow = True,
                      colors = colors,
                 s tartangle = 90,
               wedgeprops = wp,
                  textprops = dict(size = 12, color = "blue"))

# Thêm legend
ax.legend(wedges, cars,
          title = "Cars",
          loc = "center left",
          bbox_to_anchor = (1, 0, 0.5, 1))
plt.setp(autotexts, size = 8, weight = "bold")
ax.set_title("Percentage of cas sold in the first quarter")
plt.show()
```

LAB 6: Box Plot

Bài 1: Tạo Danh sách các mảng là đầu vào bắt buộc để tạo boxplot.

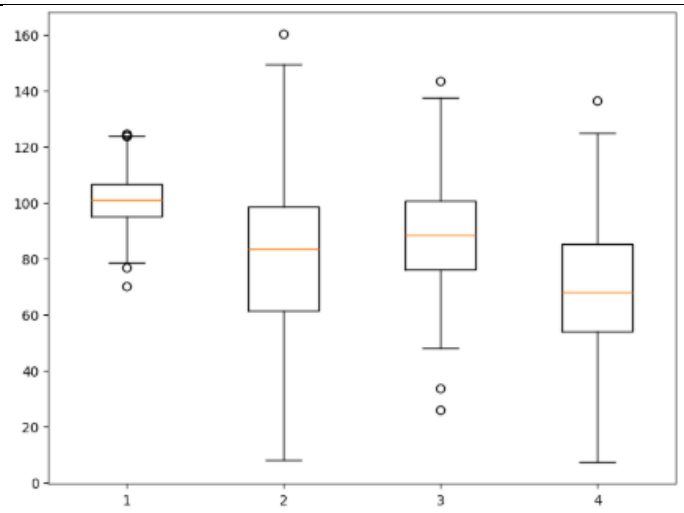
```
import numpy as np
import matplotlib.pyplot as plt

# Tạo mảng ngẫu nhiên
np.random.seed(10)
collech_1 = np.random.normal(100, 10, 200)
collech_2 = np.random.normal(80, 30, 200)
collech_3 = np.random.normal(90, 20, 200)
collech_4 = np.random.normal(70, 25, 200)

fig = plt.figure()

# Tạo 1 ax
ax = fig.add_axes([0,0,1,1])

# Tạo boxplot
bp = ax.boxplot([collech_1, collech_2,
collech_3, collech_4])
plt.show()
```



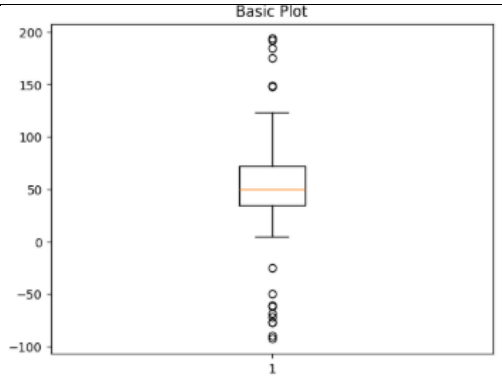
Bài 2: Biểu đồ Box plot cơ bản:

```
import numpy as np
import matplotlib.pyplot as plt

# Trạng thái ngẫu nhiên có khả năng tái tạo
np.random.seed(19680801)

# Tạo dữ liệu ngẫu nhiên
spread = np.random.rand(50) * 100
center = np.ones(25) * 50
flier_high = np.random.rand(10) * 100 + 100
flier_low = np.random.rand(10) * -100

data = np.concatenate((spread, center, flier_high, flier_low))
fig1, ax1 = plt.subplots()
ax1.set_title('Basic Plot')
ax1.boxplot(data)
plt.show()
```



Bài 3: Sử dụng hàm boxplot() của seaborn. Nhìn vào biểu đồ, sẽ dễ dàng kết luận rằng nhóm 'C' có giá trị cao hơn các nhóm khác. Tuy nhiên, ta không thể thấy sự phân bố cơ bản của các dấu chấm trong mỗi nhóm, cũng như số lượng quan sát cho mỗi nhóm.

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import pandas as pd

# Data
a = pd.DataFrame({'group': np.repeat('A', 500), 'value': np.random.normal(10, 5, 500)})
b = pd.DataFrame({'group': np.repeat('B', 500), 'value': np.random.normal(13, 1.2, 500)})
c = pd.DataFrame({'group': np.repeat('B', 500), 'value': np.random.normal(18, 1.2, 500)})
d = pd.DataFrame({'group': np.repeat('C', 20), 'value': np.random.normal(25, 4, 20)})
e = pd.DataFrame({'group': np.repeat('D', 100), 'value': np.random.uniform(12, size=100)})
df=a.append(b).append(c).append(d).append(e)

# Biểu diễn boxplot
sns.boxplot(x='group', y='value', data=df)
plt.show()
```

Bài 4:Sử dụng jupyter, colab (Cài đặt: ! pip install names):

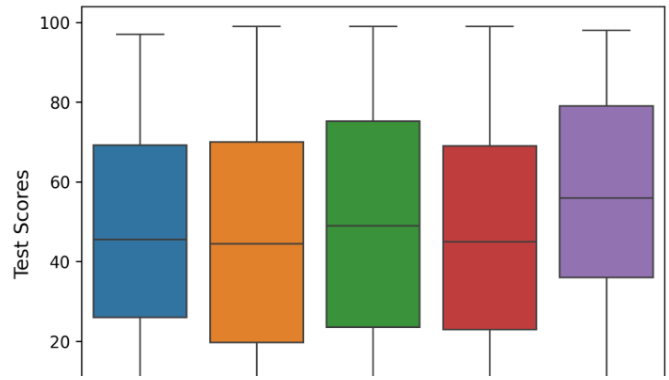
Phân phối điểm kiểm tra cho tất cả các môn học trong một biểu đồ với dữ liệu ngẫu nhiên

student_names	Literature	Chemistry	Biology	History	Geography
Jeanette Rutledge	98	50	34	3	63
Nancy Jackson	94	8	28	3	70
David Korte	6	81	88	99	62
Nancy Davis	85	50	29	59	27
Jacob Benedict	56	90	80	25	90
...	...	...	...	...	...
Mark Bowman	4	50	15	74	1
Bruce Henderson	61	85	73	25	18
Dorothy Jones	74	46	88	92	26
Grant Martinez	50	34	18	89	29
John Thomas	91	90	26	78	96

100 rows × 5 columns

```
! pip install names

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import plotly.express as px
import names
```




```
# Tạo dữ liệu giả cho điểm kiểm tra.
# Điểm số nằm trong khoảng từ 0 đến 100 và có 100 học sinh
# Tham gia kỳ thi cho 5 môn học khác nhau.
df = pd.DataFrame(np.random.randint(0,100,size = (100,5)),
                  columns =['Literature', 'Chemistry', 'Biology', 'History', 'Geography'])
# Tạo một cột có tên là 'student_names' chứa các tên được tạo ngẫu nhiên
# Và đặt cột đó làm chỉ mục.
df['student_names'] = [names.get_full_name() for i in range(100)]
df.set_index('student_names', inplace = True)

# Sử dụng gói seaborn
ax = sns.boxplot(data=df)

plt.rcParams["figure.dpi"] = 300
plt.xlabel("Subjects", size=12)
plt.ylabel("Test Scores", size=12)
plt.show()
```

LAB 7: Plotting Power Spectral Density (PSD)

Bài 1: Vẽ dao động tần số của hàm sin và cos:

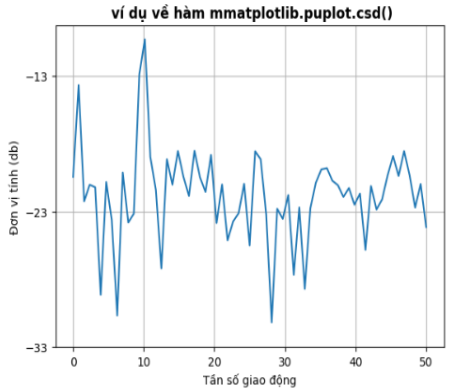
```
import numpy as np
import matplotlib.pyplot as plt

dt = 0.01
t = np.arange(0, 30, dt)
nse1 = np.random.randn(len(t))
nse2 = np.random.randn(len(t))

s1 = 1.5 * np.sin(2 *np.pi * 10 * t) + nse1
s2 = np.cos(np.pi * t) + nse2

plt.csd(s1, s2**2, 128, 1./dt)
plt.xlabel('Tần số giao động')
plt.ylabel('Đơn vị tính (db)')

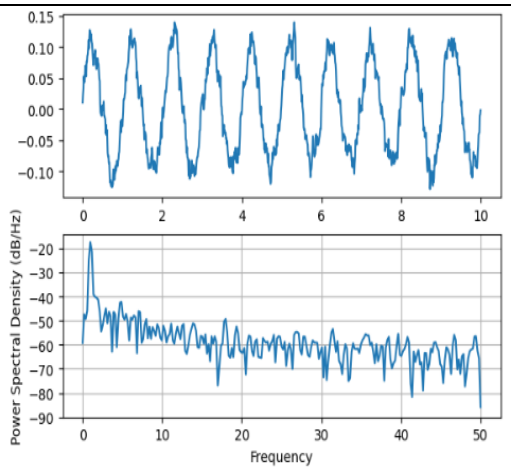
plt.title(' ví dụ về hàm mmatplotlib.puplot.csd()',fontweight ="bold")
plt.show()
```



Bài 2: Thực hiện thêm subplot:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(19680801)
dt = 0.01
t = np.arange(0, 10, dt)
nse1 = np.random.randn(len(t))
r = np.exp(-t / 0.05)
cnse = np.convolve(nse1, r) * dt
cnse = cnse[:len(t)]
s = 0.1 * np.sin(2 * np.pi * t) + cnse

plt.subplot(211)
plt.plot(t, s)
plt.subplot(212)
plt.psd(s, 512, 1 / dt)
plt.show()
```



Bài 3: Thư viện scipy.signal:

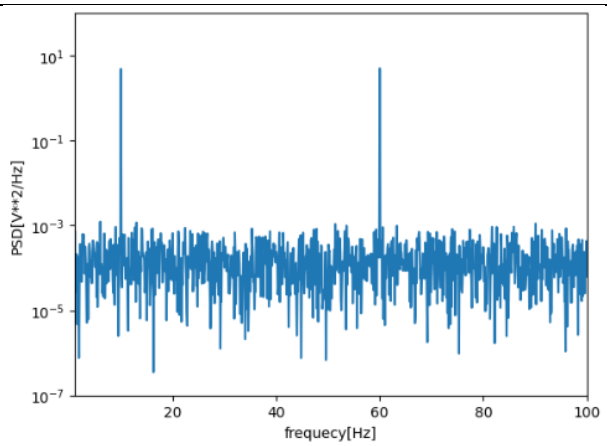
- **Trước khi đi vào tính toán PSD thực tế, chúng ta cần tạo một số dữ liệu thử nghiệm. Sử dụng hai sóng hình sin ở tần số 10Hz và 60Hz. Gói phân tích tín hiệu đi tới Scipy có triển khai tính toán các biểu đồ chu kỳ có sẵn scipy.signal.periodogram. Sử dụng chúng có thể dễ dàng tính toán mật độ phổ công suất.**
- **Sử dụng Scipy rất đơn giản, tất cả những gì chúng ta cần cung cấp cho phương pháp biểu đồ chu kỳ là dữ liệu tín hiệu thực tế và tần số lấy mẫu.**
- **Những gì thu được từ phương pháp này là các thành phần tần số và mật độ công suất tương ứng.**

```
import numpy as np
import matplotlib.pyplot as plt
import scipy.signal

fs = 1000.0
F1 = 10
F2 = 60
T = 10
N0 = -10
t = np.r_[ 0:T:(1/fs)]

signal = np.sin(2 * np.pi * F1 * t) + np.sin(2 * np.pi * F2 * t)
signal += np.random.randn(len(signal))*10**(N0/20.0)

(f,s) = scipy.signal.periodogram(signal, fs, scaling='density')
plt.semilogy(f,s)
plt.ylim([1e-7, 1e2])
plt.xlim([1, 100])
plt.xlabel('frequecy[Hz]')
plt.ylabel('PSD[V**2/Hz]')
plt.show()
```

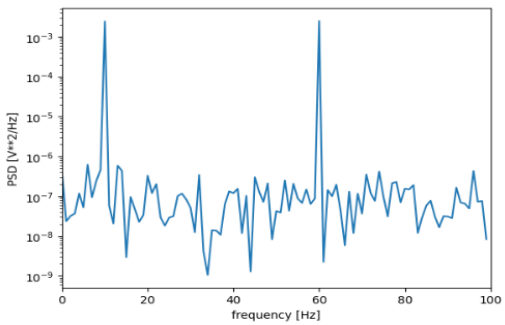


Bài 4: Triển khai Naive Python:

- **Tính toán PSD cho tần số [0,…100]**

```
import numpy as np
def Sxx(f, signal, Fs):
    t = 1/Fs
    T = len(signal)
    s = np.sum([signal[i] * np.exp(-1j*2*np.pi*f*i*t) for i in
range(T)])

return t**2 / T * np.abs(s)**2
```



```
S = [Sxx(f, signal, fs) for f in range(100)]

plt.semilogy([f for f in range(100)], S)
plt.xlim([0, 100])
plt.xlabel('frequency [Hz]')
plt.ylabel('PSD [V**2/Hz]')
plt.show()
```

LAB 8: A histogram in Matplotlib

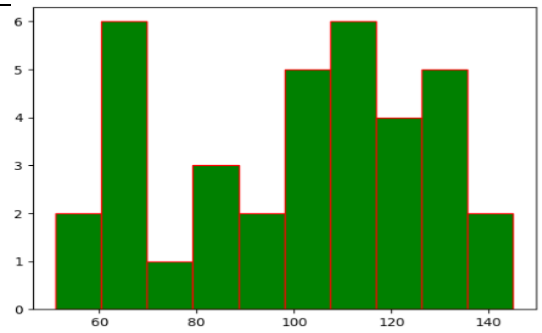
Bài Tập: Cho dữ liệu:

```
data = [87, 53, 66, 61, 67, 68, 62, 110,
        104, 61, 111, 123, 117, 119, 116,
        104, 92, 111, 90, 103, 81, 80, 101,
        51, 79, 107, 110, 129, 145, 128,
        132, 135, 131, 126, 139, 110]
```

Hãy vẽ biểu đồ Histogram với dữ liệu trên theo kết quả: (cho edgecolor: biên màu, color:màu sắc)

```
import matplotlib.pyplot as plt

x = [87, 53, 66, 61, 67, 68, 62, 110,
    104, 61, 111, 123, 117, 119, 116,
    104, 92, 111, 90, 103, 81, 80, 101,
    51, 79, 107, 110, 129, 145, 128,
    132, 135, 131, 126, 139, 110]
plt.hist(x, color='green', edgecolor='red')
plt.show()
```



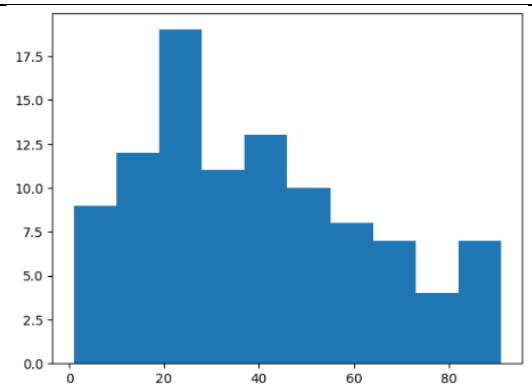
VD 1: Giả sử có dữ liệu sau về tuổi của 100 cá nhân:

```
Age

1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
75, 77, 81, 83, 84, 87, 89, 90, 90, 91
```

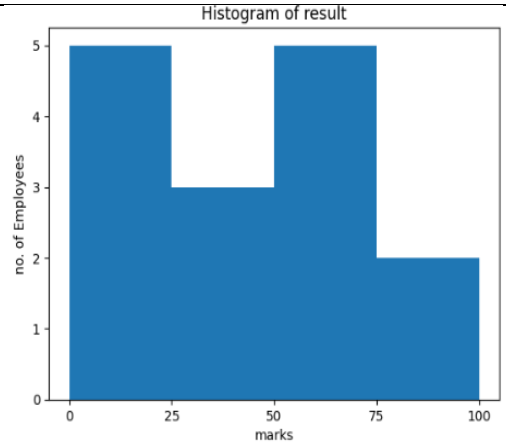
Xác định số lượng ngăn sẽ được sử dụng cho biểu đồ (đặt số lượng ngăn là 10). Dựa vào mẫu sau để vẽ biểu đồ.

```
import matplotlib.pyplot as plt
x = [1, 1, 2, 3, 3, 5, 7, 8, 9, 10,
    10, 11, 11, 13, 13, 15, 16, 17, 18, 18,
    18, 19, 20, 21, 21, 23, 24, 24, 25, 25,
    25, 25, 26, 26, 26, 27, 27, 27, 27, 27,
    29, 30, 30, 31, 33, 34, 34, 34, 35, 36,
    36, 37, 37, 38, 38, 39, 40, 41, 41, 42,
    43, 44, 45, 45, 46, 47, 48, 48, 49, 50,
    51, 52, 53, 54, 55, 55, 56, 57, 58, 60,
    61, 63, 64, 65, 66, 68, 70, 71, 72, 74,
    75, 77, 81, 83, 84, 87, 89, 90, 90, 91]
plt.hist(x, bins=10)
plt.show()
```



VD 2: Vẽ biểu đồ về đánh dấu các nhân viên trong một công ty. Bốn cột, 0-25, 26-50, 51-75 và 76-100 được xác định.

```
from matplotlib import pyplot as plt
import numpy as np
fig,ax = plt.subplots(1,1)
a = np.array([22,81,5,43,56,73,55,54,11,20,51,5,79,31,27])
ax.hist(a, bins = [0,25,50,75,100])
ax.set_title('Histogram of result')
ax.set_xticks([0,25,50,75,100])
ax.set_xlabel('marks')
ax.set_ylabel('no. of Employees')
plt.show()
```



VD 3: Sử dụng tập dữ liệu riêng biệt. Phương pháp này có thể được thực hiện khi chúng ta đã có dữ liệu riêng biệt trên bất kỳ cơ sở nào.

Ví dụ: Hãy xét 2 đối tượng tuổi là nam và nữ.

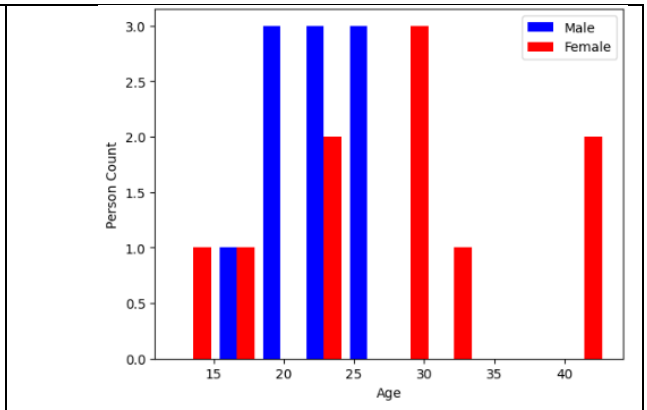
```
mens_age=18,19,20,21,22,23,24,25,26,27
female_age=22,28,30,30,12,33,41,22,43,18
```

Nếu chúng ta có loại dữ liệu riêng biệt như vậy thì chúng ta có thể chỉ định các màu khác nhau cho các nhóm khác nhau trong khi vẽ biểu đồ tần suất, tức là trong phương thức hist.

```
import matplotlib.pyplot as plt

mens_age = 18, 19, 20, 21, 22, 23, 24, 25, 26, 27
female_age = 22, 28, 30, 30, 12, 33, 41, 22, 43, 18

plt.hist([mens_age, female_age], color=['blue', 'red'],
label=['Male', 'Female'])
plt.xlabel('Age')
plt.ylabel('Person Count')
plt.legend()
plt.show()
```



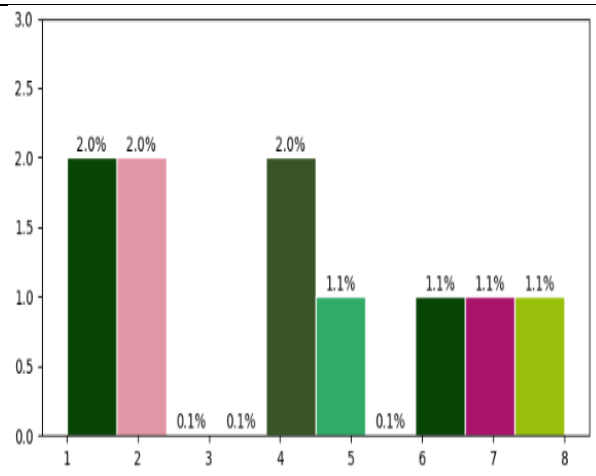
VD 4: Vẽ biểu đồ với trục Y theo tỷ lệ phần trăm trong Matplotlib:

Để vẽ một biểu đồ với trục Y dưới dạng phần trăm trong matplotlib, chúng ta có thể thực hiện các bước sau:

- **Tạo một danh sách các số như y.**
- **Tạo một số ngăn.**
- **Vẽ một biểu đồ sử dụng phương thức hist(), trong đó y, bins và edgecolor được chuyển vào đối số. Lưu trữ các bản vá để đặt tỷ lệ phần trăm trên trục Y.**
- **Tạo một danh sách các màu từ các số chữ và số đã cho.**
- **Để đặt tỷ lệ phần trăm, hãy lặp lại các bản vá (thu được ở bước 3).**
- **Đặt phạm vi đánh dấu trục Y.**
- **Để hiển thị hình, sử dụng phương thức show().**

Ví dụ:

```
import matplotlib.pyplot as plt
import numpy as np
import random
plt.rcParams["figure.figsize"] = [7.00, 3.50]
plt.rcParams["figure.autolayout"] = True
y = [4, 1, 8, 7, 6, 4, 2, 1, 2, 5]
nbins = 10
_,_, patches = plt.hist(y, bins=nbins, edgecolor='white')
colors = ["#" +
".join([random.choice('0123456789ABCDEF') for j in
range(6)])
for i in range(nbins)]
for patch in patches:
```



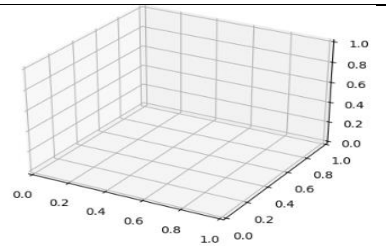
```
    patch.set_facecolor(colors[np.random.randint(100) % nbins])
    x = patch.get_x() + patch.get_width() / 2
    y = patch.get_height() + .05
    plt.annotate('{:.1f}%'.format(y), (x, y), ha='center')
plt.ylim(0, 3)
plt.show()
```

LAB 9: 3D Plot in Matplotlib

VD1: Để tạo biểu đồ 3D, dùng CÚ PHÁP sau:

```
import numpy as np
import matplotlib.pyplot as plt

fig = plt.figure()
ax = plt.axes(projection='3d')
```

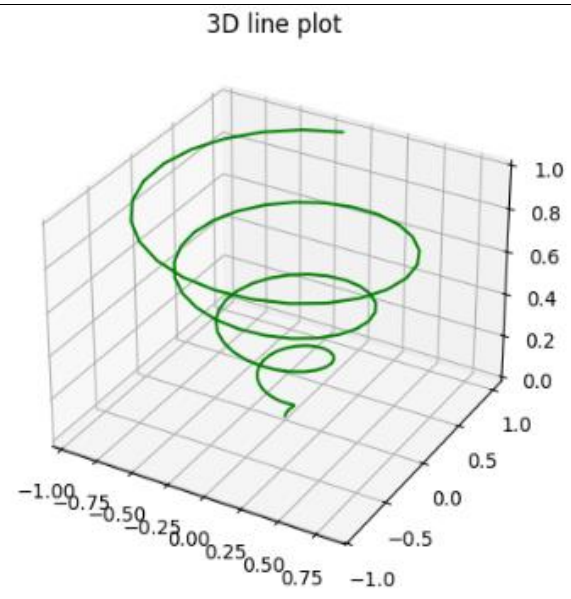


BÀI 1: Vẽ Lines and Points 3-D:

Đồ thị có đường và điểm là đồ thị 3 chiều đơn giản nhất. ax.plot3d và ax.scatter lần lượt là hàm vẽ đồ thị đường và điểm.

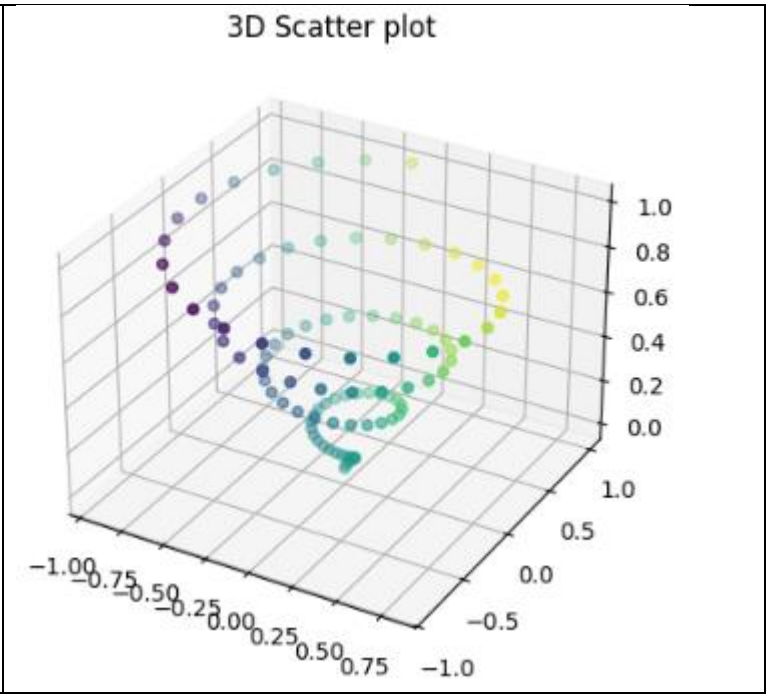
Ví dụ: Đồ thị đường 3 chiều

```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
# Cú pháp cho phép chiếu 3-D
ax = plt.axes(projection='3d')
# Xác định cả 3 trục
z = np.linspace(0, 1, 100)
x = z * np.sin(25 * z)
y = z * np.cos(25 * z)
# Plotting
ax.plot3D(x, y, z, 'green')
ax.set_title('3D line plot')
plt.show()
```



Ví dụ: 3 dimensional scattered graph

```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt
fig = plt.figure()
# Cú pháp cho phép chiếu 3-D
ax = plt.axes(projection='3d')
# Xác định trục
z = np.linspace(0, 1, 100)
x = z * np.sin(25 * z)
y = z * np.cos(25 * z)
c = x + y
ax.scatter3D(x, y, z, c=c)
# Plotting
ax.set_title('3D Scatter plot')
plt.show()
```



BÀI 2: Vẽ đồ thị Surface và Wireframes:

Biểu đồ bề mặt và biểu đồ Wireframes hoạt động trên dữ liệu dạng lưới.

Lấy giá trị lưới và vẽ trên bề mặt ba chiều.

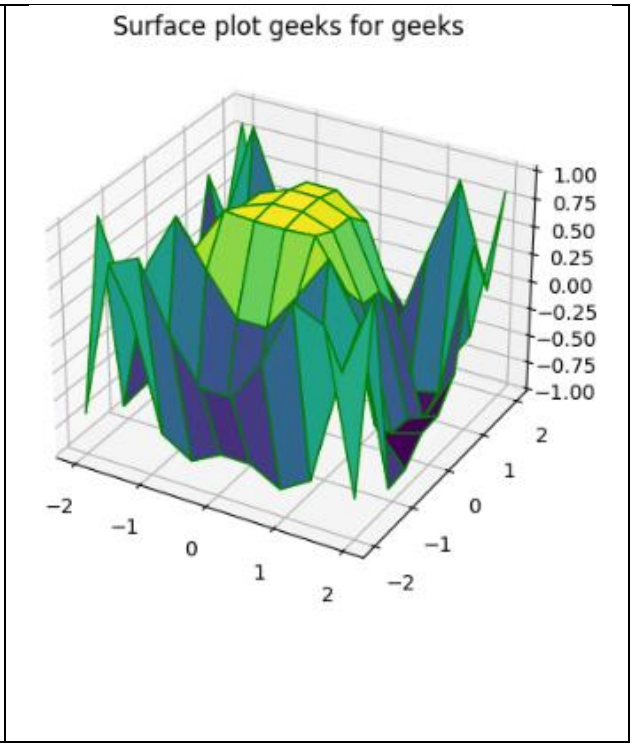
Ví dụ 1: Surface graph

```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt

# Xác định bề mặt và trục
x = np.outer(np.linspace(-2, 2, 10), np.ones(10))
y = x.copy().T
z = np.cos(x ** 2 + y ** 3)
fig = plt.figure()

# Cú pháp cho 3-D plotting
ax = plt.axes(projection='3d')

# Plotting
ax.plot_surface(x, y, z, cmap ='viridis', edgecolor ='green')
ax.set_title('Surface plot geeks for geeks')
plt.show()
```



Ví dụ 2: Wireframes

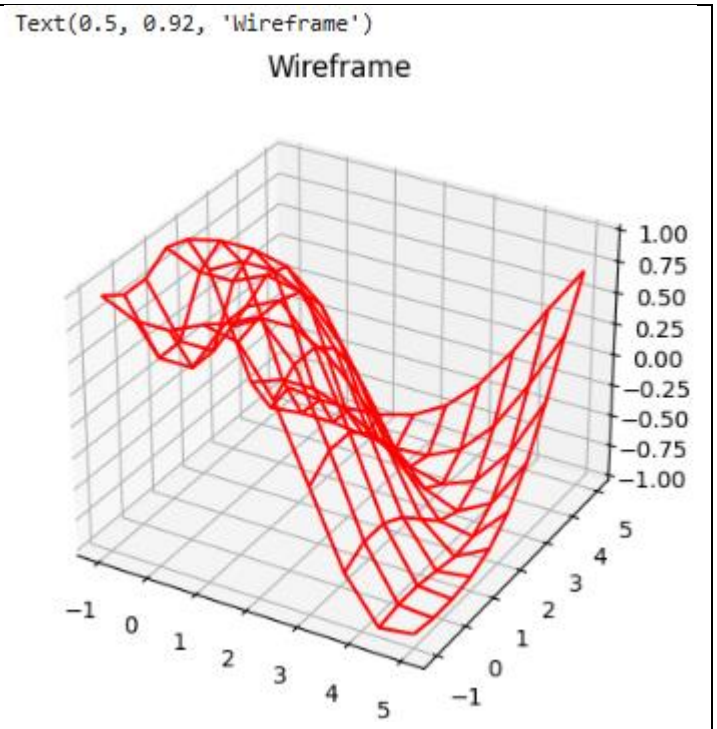
```
from mpl_toolkits import mplot3d
import numpy as np
import matplotlib.pyplot as plt

# Hàm của trục z
def f(x, y):
    return np.sin(np.sqrt(x ** 2 + y ** 2))

# Xác định trục
x = np.linspace(-1, 5, 10)
y = np.linspace(-1, 5, 10)

X,Y = np.meshgrid(x, y)
Z = f(X, Y)

fig = plt.figure()
ax = plt.axes(projection='3d')
ax.plot_wireframe(X, Y, Z, color ='red')
ax.set_title('Wireframe')
```

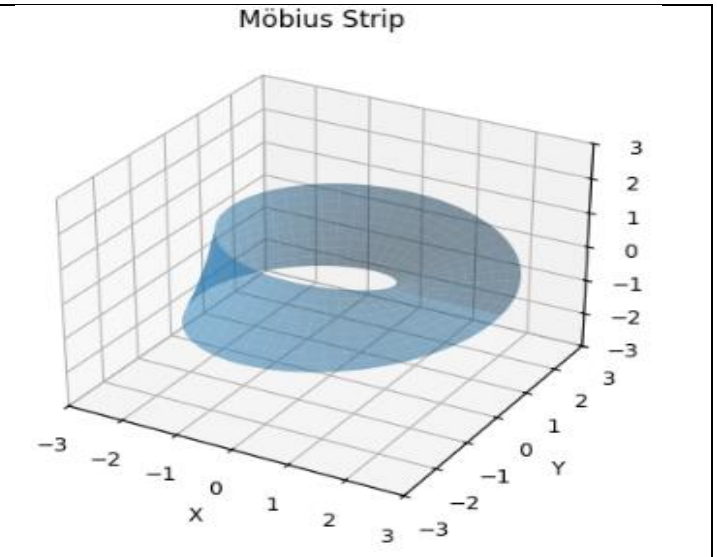


BÀI 3: Vẽ dải Möbius:

Dải Mobius còn được gọi là hình trụ xoắn, là một bề mặt- một mặt không có ranh giới. Để tạo dải Mobius, ta có tham số hóa của nó, đó là dải hai chiều và cần hai chiều nội tại.

Phạm vi góc của nó từ 0 đến 2 hình tròn quanh vòng và chiều rộng nằm trong khoảng từ -1 đến 1.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
# Xác định các tham số của Möbius
R = 2
# Định nghĩa bề mặt Möbius strip
u = np.linspace(0, 2 * np.pi, 100)
v = np.linspace(-1, 1, 10)
u, v = np.meshgrid(u, v)
x = (R + v * np.cos(u/2)) * np.cos(u)
y = (R + v * np.cos(u/2)) * np.sin(u)
z = v * np.sin(u/2)
```




```
# Tạo plot
```

```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Bề mặt của Plot the Möbius strip
ax.plot_surface(x, y, z, alpha=0.5)

# Set thuộc tính plot
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Möbius Strip')

# Set giới hạn plot
ax.set_xlim([-3, 3])
ax.set_ylim([-3, 3])
ax.set_zlim([-3, 3])

# Hiển thị plot
plt.show()
```

Ôn Tập 1:

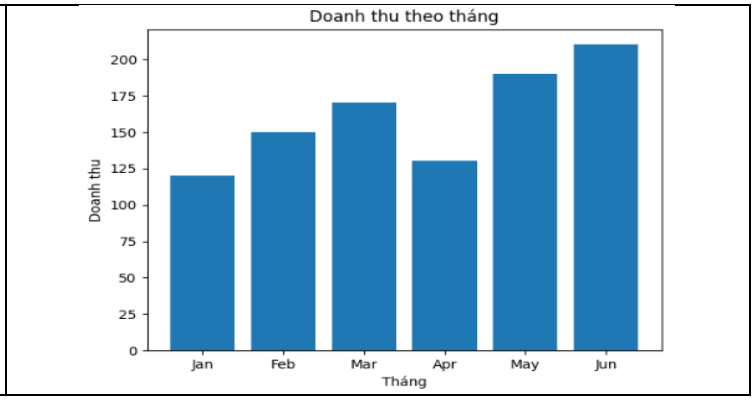
Bài 1: Column Chart – Doanh thu theo tháng:

Yêu cầu: Vẽ biểu đồ cột thể hiện doanh thu của một cửa hàng theo từng tháng trong năm.

Dữ liệu mẫu: months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']

sales = [120, 150, 170, 130, 190, 210]

```
import matplotlib.pyplot as plt
import numpy as np
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
sales = [120, 150, 170, 130, 190, 210]
plt.bar(months, sales)
plt.xlabel('Tháng')
plt.ylabel('Doanh thu')
plt.title('Doanh thu theo tháng')
plt.show()
```



Bài 2: Radar Chart – Kỹ năng cá nhân:

Yêu cầu: Vẽ biểu đồ radar thể hiện điểm số của một học sinh về 6 kỹ năng: Toán, Lý, Hóa, Văn, Anh, Tin.

Dữ liệu mẫu: labels = ['Toán', 'Lý', 'Hóa', 'Văn', 'Anh', 'Tin']

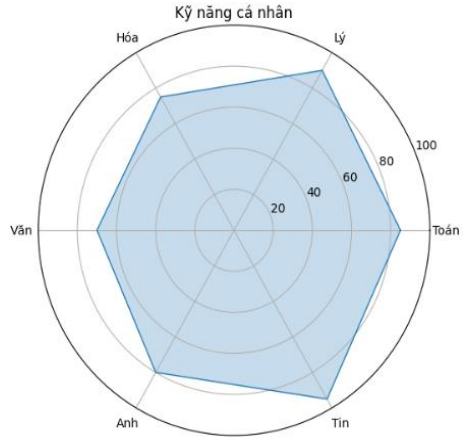
scores = [85, 90, 75, 70, 80, 95]

```
import matplotlib.pyplot as plt
import numpy as np

labels = ['Toán', 'Lý', 'Hóa', 'Văn', 'Anh', 'Tin']
scores = [85, 90, 75, 70, 80, 95]

angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(6, 6), subplot_kw=dict(polar=True))
ax.plot(angles, scores + scores[:1], linewidth=1, linestyle='solid')
ax.fill(angles, scores + scores[:1], alpha=0.25)
ax.set_thetagrids(np.degrees(angles[:-1]), labels)
ax.set_ylim(0, 100)
plt.title('Kỹ năng cá nhân')
plt.show()
```



Bài 3: Bar Chart – Dân số các thành phố:

Yêu cầu: Vẽ biểu đồ bar ngang thể hiện dân số của các thành phố.

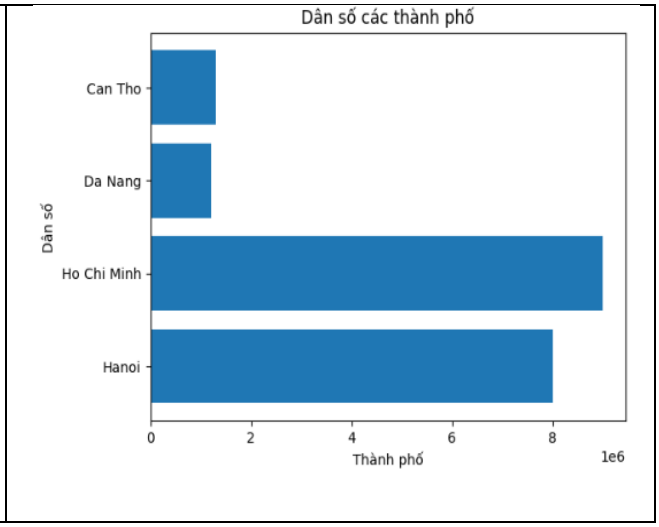
Dữ liệu mẫu: cities = ['Hanoi', 'Ho Chi Minh', 'Da Nang', 'Can Tho']

population = [8000000, 9000000, 1200000, 1300000]

```
import matplotlib.pyplot as plt
import numpy as np

cities = ['Hanoi', 'Ho Chi Minh', 'Da Nang', 'Can Tho']
population = [8000000, 9000000, 1200000, 1300000]

plt.barh(cities, population)
plt.xlabel('Thành phố')
plt.ylabel('Dân số')
plt.title('Dân số các thành phố')
plt.show()
```



Bài 4: Line Chart – Nhiệt độ theo ngày:

Yêu cầu: Vẽ biểu đồ đường thể hiện nhiệt độ trung bình trong 7 ngày.

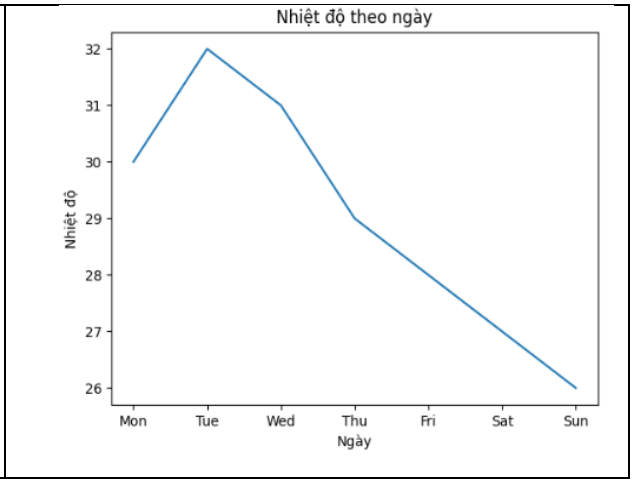
Dữ liệu mẫu: days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

temp = [30, 32, 31, 29, 28, 27, 26]

```
import numpy as np
import matplotlib.pyplot as plt

days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
temp = [30, 32, 31, 29, 28, 27, 26]

plt.plot(days, temp)
plt.xlabel('Ngày')
plt.ylabel('Nhiệt độ')
plt.title('Nhiệt độ theo ngày')
plt.show()
```



Bài 5: Area Chart – Lưu lượng truy cập website:

Yêu cầu: Vẽ biểu đồ vùng thể hiện số lượng người truy cập website trong 7 ngày.

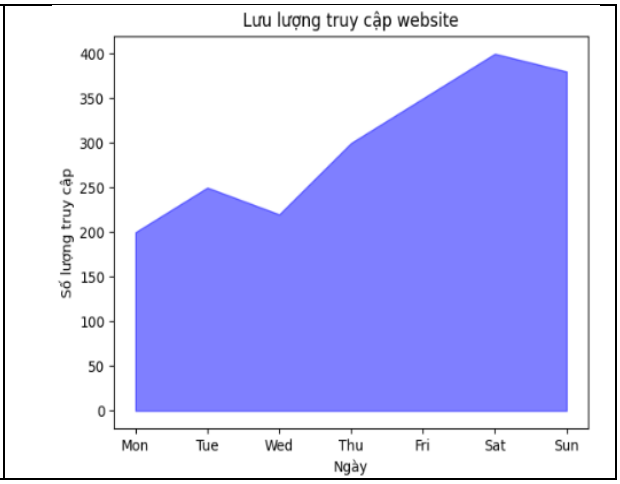
Dữ liệu mẫu: days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']

visits = [200, 250, 220, 300, 350, 400, 380]

```
import matplotlib.pyplot as plt
import numpy as np

days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
visits = [200, 250, 220, 300, 350, 400, 380]

plt.fill_between(days, visits, color='blue', alpha=0.5)
plt.xlabel('Ngày')
plt.ylabel('Số lượng truy cập')
plt.title('Lưu lượng truy cập website')
plt.show()
```



Bài 6: Pie Chart – Tỷ lệ người dùng thiết bị:

Yêu cầu:Vẽ biểu đồ tròn thể hiện tỷ lệ người dùng các loại thiết bị truy cập website.

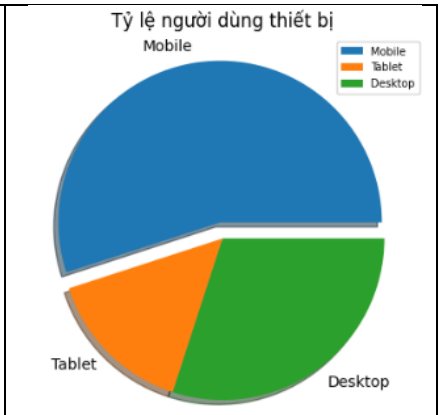
Dữ liệu mẫu:devices = ['Mobile', 'Tablet', 'Desktop']

percent = [55, 15, 30]

```
import matplotlib.pyplot as plt

devices = ['Mobile', 'Tablet', 'Desktop']
percent = [55, 15, 30]
myexplode = [0.1, 0, 0, ]

plt.pie(percent, labels=devices, explode = myexplode, shadow = True )
plt.legend(loc='best', fontsize=7)
plt.title('Tỷ lệ người dùng thiết bị')
plt.show()
```



Bài 7: Scatter Plot – Chiều cao vs Cân nặng:

Yêu cầu: Vẽ biểu đồ phân tán thể hiện mối quan hệ giữa chiều cao và cân nặng của 10 người.

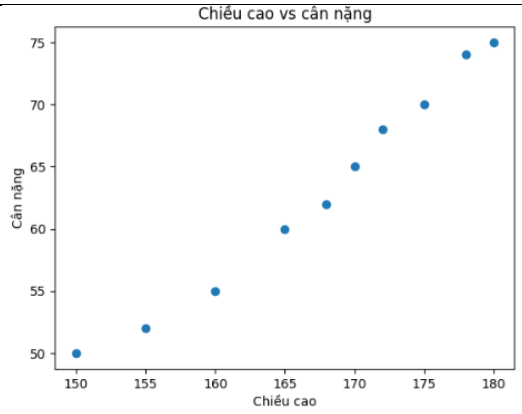
Dữ liệu mẫu: height = [150, 160, 165, 170, 175, 180, 155, 168, 172, 178]

weight = [50, 55, 60, 65, 70, 75, 52, 62, 68, 74]

```
import matplotlib.pyplot as plt
import numpy as np

height = [150, 160, 165, 170, 175, 180, 155, 168, 172, 178]
weight = [50, 55, 60, 65, 70, 75, 52, 62, 68, 74]

plt.scatter(height, weight)
plt.xlabel('Chiều cao')
plt.ylabel('Cân nặng')
plt.title('Chiều cao vs cân nặng')
plt.show()
```



Bài 8: Bubble Chart – Mức độ tiêu dùng:

Yêu cầu:

Vẽ biểu đồ bubble thể hiện mức chi tiêu của 6 khách hàng. Trục X: thu nhập, trục Y: chi tiêu, kích thước bong bóng: số giao dịch.

Dữ liệu mẫu:

income = [3000, 4000, 5000, 4500, 6000, 5500]

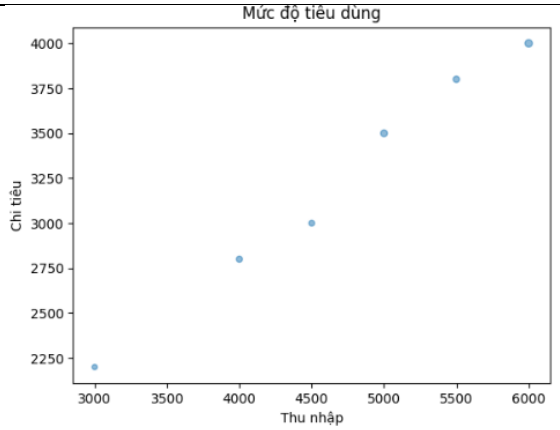
spending = [2200, 2800, 3500, 3000, 4000, 3800]

transactions = [15, 20, 25, 18, 30, 22] # kích thước bong bóng

```
import numpy as np
import matplotlib.pyplot as plt
```

```
income = [3000, 4000, 5000, 4500, 6000, 5500]
spending = [2200, 2800, 3500, 3000, 4000, 3800]
transactions = [15, 20, 25, 18, 30, 22]

plt.scatter(income, spending, s=transactions, alpha=0.5)
plt.xlabel('Thu nhập')
plt.ylabel('Chi tiêu')
plt.title('Mức độ tiêu dùng')
plt.show()
```



Ôn Tập 2:

Bài 1. Pyramid Chart (Biểu đồ kim tự tháp)

Đề bài: Viết chương trình vẽ biểu đồ kim tự tháp dân số cho một quốc gia với dữ liệu dân số nam và nữ theo các nhóm tuổi.

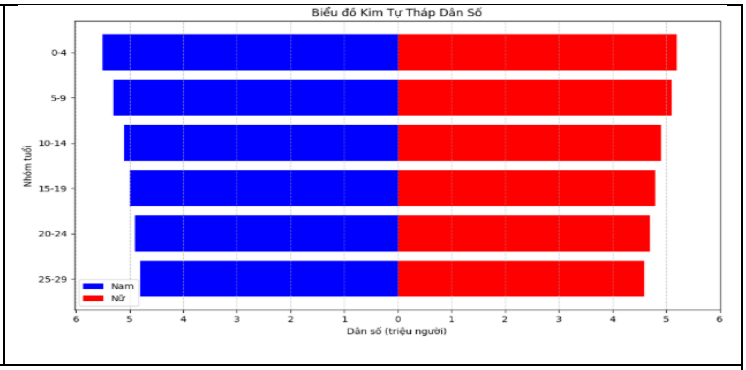
Dữ liệu: age_groups = ['0-4', '5-9', '10-14', '15-19', '20-24', '25-29']

male_population = [5.5, 5.3, 5.1, 5.0, 4.9, 4.8]

female_population = [5.2, 5.1, 4.9, 4.8, 4.7, 4.6]

```
import matplotlib.pyplot as plt

# Dữ liệu
age_groups = ['0-4', '5-9', '10-14', '15-19', '20-24',
'25-29']
male_population = [5.5, 5.3, 5.1, 5.0, 4.9, 4.8]
female_population = [5.2, 5.1, 4.9, 4.8, 4.7, 4.6]
```




```
# Đảo ngược danh sách để vẽ từ trẻ đến già theo trục y
age_groups = age_groups[::-1]
male_population = male_population[::-1]
female_population = female_population[::-1]

# Thiết lập biểu đồ
fig, ax = plt.subplots(figsize=(10, 6))

# Vẽ biểu đồ ngang: Nam bên trái (số âm), Nữ bên phải
ax.barh(age_groups, [-x for x in male_population], color='blue', label='Nam')
ax.barh(age_groups, female_population, color='red', label='Nữ')
```

```
# Thêm tiêu đề và nhãn
ax.set_title('Biểu đồ Kim Tự Tháp Dân Số')
ax.set_xlabel('Dân số (triệu người)')
ax.set_ylabel('Nhóm tuổi')

# Định dạng trục x để hiện số dương
xticks = list(range(-6, 7, 1))
ax.set_xticks(xticks)
ax.set_xticklabels([abs(x) for x in xticks])

# Thêm đường lưới và chú thích
ax.grid(True, axis='x', linestyle='--', alpha=0.7)
ax.legend()

# Hiển thị biểu đồ
plt.tight_layout()
plt.show()
```

Bài 2. Bubble Chart (Biểu đồ bong bóng)

Đề bài: Viết chương trình thể hiện dữ liệu GDP, dân số và tuổi thọ của các quốc gia dưới dạng biểu đồ bong bóng.

Dữ liệu: countries = ['Vietnam', 'USA', 'Japan']

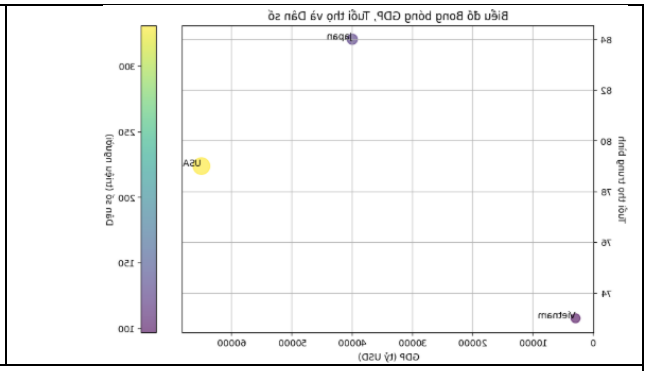
gdp = [3000, 65000, 40000]

life_expectancy = [73, 79, 84]

population = [97, 331, 125] # triệu người

```
import matplotlib.pyplot as plt

# Dữ liệu
countries = ['Vietnam', 'USA', 'Japan']
gdp = [3000, 65000, 40000]
life_expectancy = [73, 79, 84]
population = [97, 331, 125]  # triệu người
```




```
# Vẽ biểu đồ bong bóng
plt.figure(figsize=(10, 6))
plt.scatter(gdp, life_expectancy, s=population, alpha=0.6, c=population, cmap='viridis')

# Thêm chú thích cho mỗi bong bóng
for i, country in enumerate(countries):
    plt.annotate(country, (gdp[i], life_expectancy[i]))
# Thêm tiêu đề và nhãn
plt.title('Biểu đồ Bong bóng GDP, Tuổi thọ và Dân số')
```

```
plt.xlabel('GDP (tỷ USD)')
plt.ylabel('Tuổi thọ trung bình')

# Thêm thanh màu
plt.colorbar(label='Dân số (triệu người)')

# Hiển thị biểu đồ
plt.grid(True)
plt.show()
```

Bài 3. Error Chart (Biểu đồ sai số)

Đề bài: Viết chương trình hiển thị biểu đồ cột với sai số ±5% trên mỗi cột, thể hiện giá trị trung bình điểm kiểm tra của các lớp học.

Dữ liệu: classes = ['Class A', 'Class B', 'Class C']

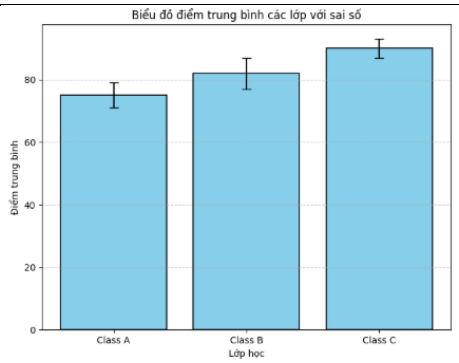
average_scores = [75, 82, 90]

errors = [4, 5, 3]

```
import matplotlib.pyplot as plt

# Dữ liệu
classes = ['Class A', 'Class B', 'Class C']
average_scores = [75, 82, 90]
errors = [4, 5, 3]

# Tính toán sai số trên và dưới
upper_errors = [score + error for score, error in zip(average_scores, errors)]
lower_errors = [score - error for score, error in zip(average_scores, errors)]
```




```
# Vẽ biểu đồ cột với sai số
plt.figure(figsize=(8, 6))
plt.bar(classes, average_scores, yerr=[
        [score - lower for score, lower in zip(average_scores, lower_errors)],
        [upper - score for upper, score in zip(upper_errors, average_scores)]
        ],
       capsize=5, color='skyblue', edgecolor='black', linewidth=1.2)

# Thêm tiêu đề và nhãn
plt.title('Biểu đồ điểm trung bình các lớp với sai số')
plt.xlabel('Lớp học')
plt.ylabel('Điểm trung bình')

# Hiển thị biểu đồ
```

```
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```

Bài 4. Heatmap (Bản đồ nhiệt)

Đề bài: Tạo bản đồ nhiệt thể hiện điểm kiểm tra của học sinh trong các môn học.

Dữ liệu: import numpy as np

students = ['Anna', 'Ben', 'Chris']

subjects = ['Math', 'English', 'Science']

scores = np.array([[90, 85, 88], [78, 92, 84], [85, 80, 86]])

```
import matplotlib.pyplot as plt
import numpy as np

# Dữ liệu
students = ['Anna', 'Ben', 'Chris']
subjects = ['Math', 'English', 'Science']
scores = np.array([[90, 85, 88], [78, 92, 84], [85, 80, 86]])

# Tạo bản đồ nhiệt
plt.imshow(scores, cmap='viridis', interpolation='nearest')

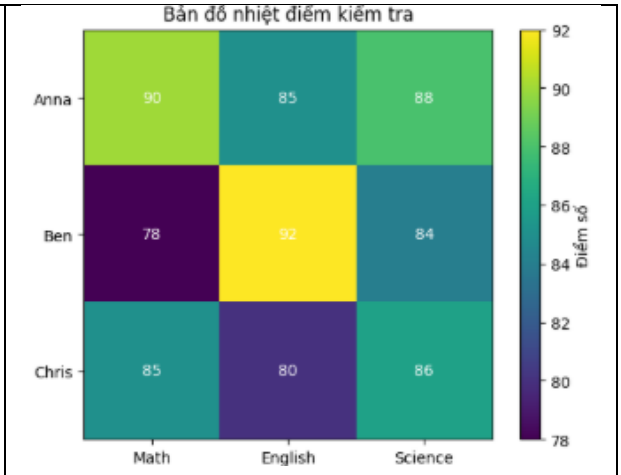
# Thêm nhãn cho trục
plt.xticks(np.arange(len(subjects)), subjects)
plt.yticks(np.arange(len(students)), students)

# Thêm thanh màu
plt.colorbar(label='Điểm số')

# Hiển thị điểm số trên bản đồ nhiệt
for i in range(len(students)):
    for j in range(len(subjects)):
        plt.text(j, i, scores[i, j], ha='center', va='center',
color='w')

# Thêm tiêu đề
plt.title('Bản đồ nhiệt điểm kiểm tra')

# Hiển thị biểu đồ
plt.show()
```



Bài 5. Gantt Chart (Biểu đồ Gantt)

Đề bài: Tạo biểu đồ Gantt thể hiện tiến độ thực hiện các giai đoạn của một dự án học sinh.

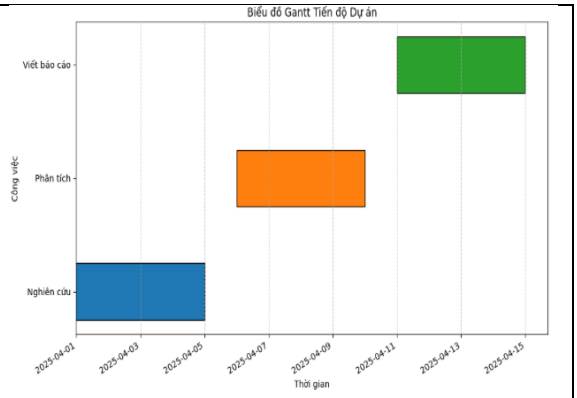
Dữ liệu: tasks = [dict(Task="Nghiên cứu", Start='2025-04-01', Finish='2025-04-05'),

dict(Task="Phân tích", Start='2025-04-06', Finish='2025-04-10'),

dict(Task="Viết báo cáo", Start='2025-04-11', Finish='2025-04-15')]

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from datetime import datetime, timedelta

# Dữ liệu
tasks = [
    dict(Task="Nghiên cứu", Start='2025-04-01', Finish='2025-04-05'),
    dict(Task="Phân tích", Start='2025-04-06', Finish='2025-04-10'),
    dict(Task="Viết báo cáo", Start='2025-04-11', Finish='2025-04-15') ]
```




```
# Chuyển đổi chuỗi ngày tháng thành đối tượng datetime
for task in tasks:
    task['Start'] = datetime.strptime(task['Start'], '%Y-%m-%d')
    task['Finish'] = datetime.strptime(task['Finish'], '%Y-%m-%d')

# Tạo biểu đồ Gantt
fig, ax = plt.subplots(figsize=(10, 6))

for task in tasks:
    duration = task['Finish'] - task['Start']
    ax.barh(task['Task'], duration.days, left=task['Start'], height=0.5, align='center', edgecolor='black')

# Định dạng trục thời gian
ax.xaxis_date()
date_format = mdates.DateFormatter('%Y-%m-%d')
ax.xaxis.set_major_formatter(date_format)
fig.autofmt_xdate()

# Thêm tiêu đề và nhãn
ax.set_title('Biểu đồ Gantt Tiến độ Dự án')
ax.set_xlabel('Thời gian')
ax.set_ylabel('Công việc')
ax.grid(True, axis='x', linestyle='--', alpha=0.7)

# Hiển thị biểu đồ
plt.tight_layout()
plt.show()
```

Bài 6. Circular Gauge (Đồng hồ đo tròn)

Đề bài: Viết chương trình thể hiện mức độ hoàn thành công việc dưới dạng đồng hồ đo tròn (gauge).

Dữ liệu: completion = 72 # phần trăm hoàn thành

```
import matplotlib.pyplot as plt
import numpy as np

# Dữ liệu đầu vào
completion = 72  # phần trăm hoàn thành

# Tạo biểu đồ với hệ tọa độ polar (tròn)
fig, ax = plt.subplots(figsize=(8, 5),
subplot_kw={'projection': 'polar'})

# Chuyển completion thành radian (0–π)
theta = (completion / 100) * np.pi
```




```
# Vẽ nền nửa vòng tròn (xám)
ax.barh(1, np.pi, left=0, height=0.4, color='#dddddd')

# Vẽ phần hoàn thành (màu xanh lá)
ax.barh(1, theta, left=0, height=0.4, color='#00cc66')

# Vẽ kim chỉ mức độ hoàn thành (màu đỏ)
ax.plot([theta, theta], [0, 1.1], color='red', linewidth=2)

# Chỉnh hướng bắt đầu từ trái sang phải
ax.set_theta_zero_location("W")
ax.set_theta_direction(-1)

# Ẩn trục
ax.set_xticks([])
ax.set_yticks([])
ax.spines['polar'].set_visible(False)

# Thêm số % bằng hệ Cartesian, không dùng trục polar
fig.text(0.5, 0.15, f"{completion}%", ha='center', va='center',
        fontsize=28, fontweight='bold', color='black')

# Tiêu đề
plt.title("Đồng hồ đo nửa hình tròn", fontsize=16, pad=20)
plt.tight_layout()
plt.show()
```